



National University of Computer and Emerging Sciences, Lahore



FYP Report Title

Name of Student 1 Roll No. 1 BS(CS/DS/SE)

Name of Student 2 Roll No. 2 BS(CS/DS/SE)

Name of Student 3 Roll No. 3 BS(CS/DS/SE)

Supervisor: SupervisorName

Final Year Project

October 9, 2026

Anti-Plagiarism Declaration

This is to declare that the above publication was produced under the:

Title: FYP Report Title

is the sole contribution of the author(s), and no part hereof has been reproduced as it is the basis (cut and paste) that can be considered Plagiarism. All referenced parts have been used to argue the idea and cited properly. I/We will be responsible and liable for any consequence if a violation of this declaration is determined.

Date:

Name:

Signature:

Name:

Signature:

Name:

Signature:

Author's Declaration

This states Authors' declaration that the work presented in the report is their own, and has not been submitted/presented previously to any other institution or organization.

Abstract

An Abstract is a short summary of the work being reported. Three to five sentences describing the essence of the work. An abstract is a short, 50-125 words summary of a work. An Abstract should state your work's purpose, findings, and conclusion without commenting on or evaluating the work itself. It should be only one paragraph. Put the abstract on a separate page that follows the title page.

Executive Summary

The executive summary should be one to two pages' of an overview of the information contained in the FYP report. It should give the reader an easy reference, in a very brief form, to the important information contained in the report and explained in more detail in the body of the report. People reading the report will use this section as a reference during presentations.

Table of Contents

List of Figures	ix
List of Tables	x
1 Introduction	1
1.1 Purpose of this Document	1
1.2 Intended Audience	2
1.3 Definitions, Acronyms, and Abbreviations	2
1.4 Conclusion	2
2 Project Vision	3
2.1 Problem Domain Overview	3
2.2 Problem Statement	3
2.3 Problem Elaboration	3
2.4 Goals and Objectives	3
2.5 Project Scope	3
2.6 Sustainable Development Goal (SDG)	4
2.7 Constraints	5
2.8 Business Opportunity	5
2.9 Stakeholders Description/ User Characteristics	5
2.9.1 Stakeholders Summary	5
2.9.2 Key High-Level Goals and Problems of Stakeholders	5
3 Literature Review / Related Work	6
3.1 Definitions, Acronyms, and Abbreviations	6
3.2 Detailed Literature Review	6
3.2.1 Related Research Work 1 / Related Development Project (Please make sure to add an appropriate heading here and not keep this same heading, this is your info only.)	6

3.2.2	Example LR 1: for Development Project	6
3.2.3	Example LR 2: for Research and Development Project	7
3.3	Literature Review Summary Table	8
3.4	Conclusion	8
4	Software Requirement Specifications	10
4.1	List of Features	10
4.2	Functional Requirements	10
4.3	Quality Attributes	10
4.4	Non-Functional Requirements	10
4.5	Assumptions	10
4.6	Use Cases	10
4.7	Hardware and Software Requirements	12
4.7.1	Hardware Requirements	13
4.7.2	Software Requirements	13
4.8	Graphical User Interface	13
4.9	Database Design (if required; this means if you are using noSQL, you will not provide ER but the other design element and data dictionary should still be there. Explain and elaborate your DB design)	13
4.9.1	ER Diagram	13
4.9.2	Data Dictionary	13
4.10	Risk Analysis	13
5	Proposed Approach and Methodology	14
5.1	Methodological Foundations	14
5.1.1	Masked Latent Prediction and M2D	14
5.1.2	Specializing a General Model: M2D-X	15
5.1.3	Music-Specific Self-Supervised Models	16
5.1.4	Further Pre-training, Forgetting and Regularization	17
5.1.5	Design Decisions Derived from the Literature	18
5.2	Research Questions and Hypotheses	18
5.3	Dataset Acquisition	19
5.3.1	Choice of Corpus	19
5.3.2	Archives and Metadata	20
5.3.3	Eligibility Rules and Subset Selection	20

5.3.4	Background-Noise Corpus	21
5.3.5	Licensing and Ethics	21
5.4	Data Preprocessing Pipeline	22
5.4.1	Audit and Decoding	22
5.4.2	Front-End	23
5.4.3	Cropping and Normalization	23
5.4.4	Musical-Teacher Targets	24
5.4.5	Storage, Manifest and Integrity Checks	24
5.5	Further Pre-training Framework	25
5.5.1	Overview, Initialization and Alternatives Considered	25
5.5.2	Masked Latent Prediction on Noisy Music	26
5.5.3	Background Noise and Denoising	26
5.5.4	Frozen-Teacher Regularization	27
5.5.5	Musical Teacher	27
5.5.6	Joint Objective and Optimization	28
5.5.7	Compute and Memory Budget	28
5.6	Experimental Design	29
5.6.1	Reference Models	30
5.6.2	Pre-training Arms	30
5.6.3	Controls and Reproducibility	31
5.7	Evaluation Protocol	32
5.7.1	Downstream Tasks	32
5.7.2	Frozen-Feature Linear Evaluation	32
5.7.3	Metrics, Aggregates and Statistics	33
5.7.4	Threats to Validity	34
5.8	Conclusion	34
6	High-Level and Low-Level Design	36
6.1	System Overview	36
6.2	Design Considerations	36
6.2.1	Assumptions and Dependencies	36
6.2.2	General Constraints	36
6.2.3	Goals and Guidelines	37
6.2.4	Development Methods	37
6.3	System Architecture	38

6.3.1	Subsystem Architecture	38
6.4	Architectural Strategies	39
6.4.1	Architectural Strategies Strategy-1 name or description	40
6.4.2	Architectural Strategies Strategy-2 name or description	40
6.5	Domain Model/Class Diagram	40
6.6	Policies and Tactics	40
6.6.1	Policy/tactic-1 name or description	41
6.6.2	Policy/tactic-2 name or description	41
7	Implementation and Test Cases	42
7.1	Implementation	42
7.1.1	Implementation of First Component/Algorithm	42
7.2	Test case Design and description	42
7.3	Test Metrics	43
7.3.1	Sample Test case Metric.No.2	43
8	User Manual	44
9	Experimental Results and Discussion	45
10	Conclusions	46
10.1	aa	46
10.1.1	dfs	46
A	First Appendix if Required	53
A.1	References	53
A.2	Equations	53
A.3	Figures	53
A.4	Tables	54
A.5	Pseudo Code	54
A.6	Code of Programming Languages	54
A.7	Recommendations for L ^A T _E X	55

List of Figures

2.1	This figure represents all the SDGs that can be targets of an FYP; Adding this sub-caption is mandatory; make sure you describe the picture in a way that if a visually impaired person is reading using a text reader, they can understand whats in this picture.	4
5.1	Overview of the proposed methodology	14
5.2	Data acquisition and preprocessing pipeline	22
5.3	Architecture of the further pre-training framework	25
A.1	Fast Logo is the Caption.	53

List of Tables

3.1	This summary Table is for a development project, and you must identify each column clearly. Must need at least 15 relevant studies	8
3.2	This summary Table is for a research project, and you must identify each column clearly. Must need at least 15 relevant studies	8
4.1	A sample use case is given in the above table. You have to follow this tabular format for use cases. Following is the description of the content required in this section. No separate heading is required or use case tables. Keep them all under a generic use case heading. Also, In the previous version, usecase tables dont have cpation, in this version, we do have.	11
5.1	Literature findings and the design decisions they lead to	18
5.2	FMA audio archives and their use in this study	20
5.3	Front-end and target parameters	23
5.4	Fixed settings of the further pre-training runs	28
5.5	Reference models	30
5.6	Pre-training arms	31
5.7	Downstream tasks	32
7.1	Sample Test case No.1	42
7.2	Sample Test case Matric.No.1	43
A.1	Give a Caption to the Table.	54

Chapter 1 Introduction

This chapter is mandatory. The introduction should contain a brief overview of the problem being addressed and the background information needed for the reader to understand the work being done and the reasoning behind it. The last paragraph of the introduction chapter should contain an outline of the entire report. Summarize each chapter in one line to make the last paragraph.

1.1 Purpose of this Document

Specify the purpose of this Project. The purpose of the document in an FYP report is to clearly and concisely state the main objective and goals of the project, as well as the research question(s) or hypothesis(es) being investigated. It should provide a brief overview of the background and context of the project, and explain why the research is important and relevant. The purpose of the document should be written in a way that is easy for the reader to understand, and should be included at the beginning of the report.

In addition to providing an overview of the project, the purpose of the document should also outline the specific objectives and goals of the research, such as what the project aims to achieve and what the research questions or hypotheses are. This will help the reader understand the scope of the project and what to expect from the report. It is also important to include a brief description of the methods and techniques used in the research, as well as any limitations or assumptions of the project. This will provide the reader with a clear understanding of how the research was conducted and what the results and conclusions are based on. Finally, it should be clear what the report is going to cover, what kind of information will be presented, and in what order.

For Example, "The purpose of this document is to present the design, implementation, and evaluation of our Final Year Project (FYP) which aims to develop a web-based application for small businesses to manage their inventory and customer orders. The project will focus on creating a user-friendly interface and providing a seamless user experience. The goal of this research is to provide a Minimum Viable Product (MVP) that can be used by small businesses to improve their inventory management and customer order processing. The research question we aim to answer is: Can we develop a web-based application that can improve inventory management and customer order processing for small businesses? This report will present the methodology, design, implementation, testing, and evaluation of our project, as well as the limitations and future work."

1.2 Intended Audience

Identifying and writing the intended audience for a Final Year Project (FYP) report is an important step in the writing process. The intended audience is the group of people for whom the report is written and who will primarily read and use the report. It is crucial to identify the intended audience early in the writing process as this will guide the report's style, tone, and level of detail.

To identify the intended audience, consider the report's purpose, who will be using the report and for what purpose. For example, the intended audience for a FYP report in Computer Science may be a panel of professors who will be evaluating the project. In contrast, the intended audience for a FYP report in Business may be potential investors or clients.

1.3 Definitions, Acronyms, and Abbreviations

List all important definitions, acronyms, and abbreviations used in this document. For example: **SDG**: Sustainable Development Goal

FYP: Final Year Project

MVP: Minimum Viable Product

UI: User Interface

UX: User Experience

Agile: Agile Development

SCRUM: Scrum Development

REST: Representational State Transfer

ORM: Object-Relational Mapping

CRUD: Create, Read, Update, Delete

API: Application Programming Interface

1.4 Conclusion

The last paragraph of the introduction chapter should contain an outline of the entire report. Summarize each chapter in one line to make the last paragraph.

Chapter 2 Project Vision

For each related work provide a paragraph of introduction and at the end, a paragraph of conclusions. Give a page break after the chapter ends. This chapter is mandatory. Clearly specify the goals and objectives of the project along with the scope of the project. (You can make sub-heading of goals and objectives and the scope of the project).

2.1 Problem Domain Overview

Describe in full detail what your system does. This section can be further divided into sub-sections, but basically, a paragraph description of the system is required.

2.2 Problem Statement

Provide the **problem** which you are trying to solve.

2.3 Problem Elaboration

Describe the problem in detail and the various sub-problems you would work on.

2.4 Goals and Objectives

Write goals and objectives here.

2.5 Project Scope

Specify the scope of your project here.

For example, The scope of this project is to design and develop a web-based application for small businesses to manage their inventory and customer orders. The application will allow small businesses to:

- Create, read, update, and delete inventory items
- Create, read, update, and delete customer orders
- Generate reports on inventory levels and customer orders
- Send notifications to customers about their orders
- Provide an easy-to-use user interface and a seamless user experience

The project will be developed using a combination of HTML, CSS, JavaScript, and a web-development framework such as React.js or Angular.js. The application will use a RESTful API to communicate with a back-end database, which will be implemented using a ORM library. The project will be developed using Agile development principles, and project management will be done using SCRUM.

The project deliverables will include:

- A functional web-based application for small businesses to manage their inventory and customer orders
- A user manual for the application
- A documentation of the design, implementation, and testing process

The project scope does not include the following:

- Providing hosting or server infrastructure for the application
- Developing mobile applications (iOS or Android)
- Integrating the application with third-party services or APIs

This project scope will be used as a guide throughout the development

2.6 Sustainable Development Goal (SDG)

Identify at least one SDG area that you are targeting in this project. This has to be documented as a paragraph. The complete SDG list is shown in Figure ??.



Figure 2.1: This figure represents all the SDGs that can be targets of an FYP; Adding this sub-caption is mandatory; make sure you describe the picture in a way that if a visually impaired person is reading using a text reader, they can understand what's in this picture.

2.7 Constraints

2.8 Business Opportunity

2.9 Stakeholders Description/ User Characteristics

Identify and briefly describe who the users of this system will be, and their roles.

2.9.1 Stakeholders Summary

2.9.2 Key High-Level Goals and Problems of Stakeholders

Notice how a section's heading is changed to "heading 1". It will show up in the table of contents as a section under a chapter. Notice its numbering also. Similarly, subsections can have heading 2 and heading 3 styles.

NOTE: Subsections from 2.7 to 2.9 are optional for Research projects but compulsory for Development projects.

Chapter 3 Literature Review / Related Work

For each related work, provide a paragraph of introduction and, at the end, a paragraph of conclusions. Give a page break after the chapter ends. **This chapter is mandatory.**

3.1 Definitions, Acronyms, and Abbreviations

3.2 Detailed Literature Review

This section will include a detailed literature review of your problem area. Make different categories for different types of work done in the past. In addition to textual descriptions, make a summary table that describes each paper that you have read, along with references.

3.2.1 Related Research Work 1 / Related Development Project (Please make sure to add an appropriate heading here and not keep this same heading, this is your info only.)

- Summary of the research item / Development Project (1 or 2 paragraphs)
- Critical analysis of the research item / Development Project (Strengths and Weaknesses) 1 paragraph
- Relationship to the proposed work - 1 paragraph

3.2.2 Example LR 1: for Development Project

The project ABC [1] focused on building a real-time customer service dashboard designed to streamline query management and improve response efficiency within a mid-sized software firm. The system was developed using a modular architecture, integrating ticket tracking, live chat support, and performance analytics into a single interface. Built with React for the frontend and Node.js for the backend, the dashboard also featured role-based access control and SLA monitoring tools. During testing, the dashboard demonstrated a 35% reduction in query resolution time and improved internal coordination between support agents and technical teams. The project was deployed in phases, starting with internal use and later expanding to client-facing support channels.

One of the key strengths of this development effort was its adaptability to existing workflows. The dashboard was designed to integrate seamlessly with the company's CRM and internal communication tools, minimizing disruption during rollout. Its real-time analytics allowed supervisors to monitor agent performance and customer satisfaction trends. However, the project faced limitations in scalability during peak traffic hours, and the initial version lacked multilingual support, which restricted its usability for

international clients. These issues were partially addressed in later iterations but remain areas for future enhancement.

This development project directly supports the proposed research by providing a practical foundation for evaluating customer service efficiency. While the research explores theoretical models and AI integration, the dashboard serves as a tangible implementation that can be enhanced with AI-driven features such as automated query routing and sentiment analysis. The performance metrics gathered during this project will inform the research's evaluation framework and help validate proposed improvements in service delivery and operational management.

3.2.3 Example LR 2: for Research and Development Project

Smith et al. [2] investigated the integration of AI-powered chatbots within corporate customer service frameworks, focusing on automating Tier-1 support queries. Their study involved deploying a Natural Language Processing (NLP) based chatbot across three multinational firms and evaluating its performance over a six-month period. The results demonstrated a 40% reduction in average response time and a 25% increase in customer satisfaction ratings. The chatbot was trained using historical customer interaction logs and was integrated with existing CRM systems to ensure contextual accuracy and personalized responses. Additionally, the study emphasized the scalability of chatbot systems, noting their ability to handle thousands of simultaneous queries without compromising quality, especially during peak service hours when traditional support teams struggled to maintain SLA compliance.

One of the strengths of this research lies in its empirical approach, offering quantifiable improvements in service metrics. The integration with CRM systems added depth to the chatbot's responses, enhancing the overall customer experience. However, the study's limitations include its narrow focus on Tier-1 queries, with minimal exploration of escalation protocols or emotional intelligence in handling sensitive customer issues. It also lacks discussion on the long-term adaptability of the chatbot to evolving customer expectations and organizational changes.

This research is directly relevant to the proposed work, which aims to develop a hybrid customer service model that combines AI efficiency with human oversight. While they focused primarily on automation, the proposed work seeks to extend their framework by incorporating feedback loops, escalation mechanisms, and sentiment analysis to handle more complex queries. The foundational insights from this study will inform the design and evaluation metrics of the proposed system, particularly in terms of response time, customer satisfaction, and operational scalability.

3.3 Literature Review Summary Table

Please provide a compact summary of your literature review as provided in the table 3.1, for the development project. The Table above explains a way to present your literature review for development

Table 3.1: This summary Table is for a development project, and you must identify each column clearly. Must need at least 15 relevant studies

Application	Features	Relevance	Limitations
Rapid Miner [3]	User interface design	Provides information on user requirements and design considerations	Limited sample size and geographic diversity of participants
TensorFlow [4]	Competitive analysis	Identifies potential competitors and opportunities for differentiation	Limited to apps in the same category and not considering other possible alternatives
Abc Keras [5]	Technical infrastructure	Helps to identify and plan for necessary technical resources and infrastructure	Assumes all requirements are known and does not evaluate the possibility of new requirements emerging in the future
Abc Xyz [1]	User experience testing	Provides feedback on user experience and identifies areas for improvement	Limited sample size and testing environment not fully representative of real-world conditions

projects. If you have a research project, the summary should be written in the below table 3.2 format.

Table 3.2: This summary Table is for a research project, and you must identify each column clearly. Must need at least 15 relevant studies

Author	Method	Results	Limitations
Smith et al. [2]	Case study with a sample size of 50	Positive effects of XYZ on ABC, 93.74% accuracy	Small sample size
Johnson et al. [3]	Literature review utilizing keyword-based search	A comprehensive overview of XYZ in the field of DEF, 12.8% precision	Limited to specific subfield
Williams et al. [6]	Survey of 1000 participants using a structured questionnaire	Strong correlation between XYZ and GHI, R-squared = 0.72	Self-reported data
Jones et al. [4]	Experimental design with 100 participants, Utilized SVM with the combination of PCA	XYZ significantly improves JKL in certain populations, AUC = 0.86	Limited generalizability

3.4 Conclusion

Write the conclusion of the literature review. The columns in the table depending upon your problem and should be specific to your project as described in 3.1, 3.2, respectively. Proper citations and references

are necessary. The template has already an integrated IEEE references format. Go to Google scholar, copy the bib text of your reference, and paste the reference into the fypbib.bib file. Then all you have to do is use [2].

NOTE: For development projects, describe related or similar work done by other teams and details of their methods/algorithms. For a research project, a detailed literature survey is expected.

Chapter 4 Software Requirement Specifications

Describe all modules of requirements and design in clear English text along with the necessary diagram and figures. Anyone reading your report should be able to reproduce your system/results after reading it. It describes functional requirements, design constraints, and other factors necessary to provide a complete and comprehensive description of the requirements for the software.

For each chapter, provide a paragraph of introduction and at the end, a paragraph of conclusions. Make sure no heading/subheading is blank. Write text to introduce each section as well. **NOTE: This Chapter is optional for Research projects but compulsory for Development projects.**

4.1 List of Features

List all important features of your system here.

4.2 Functional Requirements

The functional requirements fully describe the external behavior of the system. Identify and list each functionality and give a brief description, along with the user of each functionality.

4.3 Quality Attributes

4.4 Non-Functional Requirements

This section should describe all the non-functional requirements, including reusability, performance (how many maximum users can access it at a time), extensibility, etc.

4.5 Assumptions

List down all the assumptions made for the specification.

4.6 Use Cases

This section lists use cases or scenarios from the use-case model if they represent some significant, central functionality of the final system, or if they have a large architectural coverage—they exercise many architectural elements, or if they stress or illustrate a specific, delicate point of the architecture.

The table 4.1 provides a simple use case.

Table 4.1: A sample use case is given in the above table. You have to follow this tabular format for use cases. Following is the description of the content required in this section. No separate heading is required or use case tables. Keep them all under a generic use case heading. Also, In the previous version, usecase tables dont have caption, in this version, we do have.

Name	Sample Use Case Name Here		
Actors	Admin, Business Owner, Store Manager		
Summary	The user shall provide their email and password on the login form, and after successful verification, redirect the user to the home page.		
Pre-Conditions	The user must be in the database records, either added by any of the authorized users or added manually by a developer. The user must not already be logged in.		
Post-Conditions	The user’s session is successfully established and shall be redirected to the home page.		
Special Requirements	None		
Basic Flow			
Actor Action		System Response	
1	The user opens the login page.	2	The login page is displayed asking for email and password.
2	The user enters valid email and password.	4	The system verifies the email and password, establishes a session for the user and redirects the user to the home page.
Alternative Flow			
3	The user enters invalid email or password.	4-A	The system responds with an error message: Incorrect email or password entered.

This section lists use cases or scenarios from the use-case model if they represent some significant, central functionality of the final system, or if they have a large architectural coverage—they exercise many architectural elements or if they stress or illustrate a specific, delicate point of the architecture.

Summary: The description briefly conveys the role and purpose of the use case. A single paragraph will suffice for this description.

Basic Flow: This use case starts when the actor does something. An actor always initiates use cases. The use case describes what the actor does and what the system does in response. It is phrased in the form of a dialog between the actor and the system.

The use case describes what happens inside the system, but not how or why. If information is exchanged, be specific about what is passed back and forth. For example, it is not very illuminating to say that the actor enters customer information. It is better to say the actor enters the customer's name and address. A Glossary of Terms is often useful to keep the complexity of the use case manageable you may want to define things like customer information there to keep the use case from drowning in details.

Simple alternatives may be presented within the text of the use case. If it only takes a few sentences to describe what happens when there is an alternative, do it directly within the **Flow of Events** section. If the alternative flow is more complex, use a separate section to describe it. For example, an Alternative

Flow subsection explains how to describe more complex alternatives.

A picture is sometimes worth a thousand words, though there is no substitute for clean, clear prose. If it improves clarity, feel free to paste graphical depictions of user interfaces, process flows or other figures into the use case. If a flow chart is useful to present a complex decision process, by all means use it! Similarly for state-dependent behavior, a state-transition diagram often clarifies the behavior of a system better than pages upon pages of text. Use the right presentation medium for your problem, but be wary of using terminology, notations or figures that your audience may not understand. Remember that your purpose is to clarify, not obscure.

First Alternative Flow: More complex alternatives are described in a separate section, referred to in the Basic Flow subsection of Flow of Events section. Think of the Alternative Flow subsections like alternative behavior each alternative flow represents alternative behavior usually due to exceptions that occur in the main flow. They may be as long as necessary to describe the events associated with the alternative behavior. When an alternative flow ends, the events of the main flow of events are resumed unless otherwise stated.

Second Alternative Flow: There may be, and most likely will be, a number of alternative flows in a use case. Keep each alternative flow separate to improve clarity. Using alternative flows improves the readability of the use case, as well as preventing use cases from being decomposed into hierarchies of use cases. Keep in mind that use cases are just textual descriptions, and their main purpose is to document the behavior of a system in a clear, concise, and understandable way.

Special Requirements: A special requirement is typically a nonfunctional requirement that is specific to a use case, but is not easily or naturally specified in the text of the use case's event flow. Examples of special requirements include legal and regulatory requirements, application standards, and quality attributes of the system to be built including usability, reliability, performance or supportability requirements. Additionally, other requirements such as operating systems and environments, compatibility requirements, and design constraints should be captured in this section.

Pre-Conditions: A pre-condition of a use case is the state of the system that must be present prior to a use case being performed.

Post-Conditions: A post-condition of a use case is a list of possible states the system can be in immediately after a use case has finished.

4.7 Hardware and Software Requirements

List the hardware and software requirements that will be required to develop and deploy the project.

4.7.1 Hardware Requirements

4.7.2 Software Requirements

4.8 Graphical User Interface

This section should give the GUI dumps of each screen, with reference to the users. The navigation flow of each user is also required, and each GUI should mark the functionality/use case that it covers.

4.9 Database Design (if required; this means if you are using noSQL, you will not provide ER but the other design element and data dictionary should still be there. Explain and elaborate your DB design)

4.9.1 ER Diagram

4.9.2 Data Dictionary

4.10 Risk Analysis

List and explain the risks that may be encountered during the project. For e.g.: technical risks, business risks, etc.

Chapter 5 Proposed Approach and Methodology

This chapter describes the proposed approach at the level of algorithms and procedures. The study asks whether a general-purpose audio model trained by masked latent prediction can compete with music-specific foundation models after further pre-training on a small amount of music data. Figure 5.1 gives an overview. The approach has four parts: a data pipeline that acquires a 10,000-track subset of the Free Music Archive (FMA) and converts it into the exact input format of the pre-trained model; a further pre-training framework, called M2D-X-Music in this report, that combines the denoising and regularization recipe of M2D-X with a MERT-style musical teacher; an experimental design that isolates the contribution of each component and of the amount of data under a single-GPU budget; and an evaluation protocol that measures gains on music tasks, retention of general-audio ability, and the remaining gap to music-specific reference models. Section 5.1 first summarizes the literature that motivates each design choice. No results are reported in this chapter: it specifies what will be run and how the outcome will be judged.

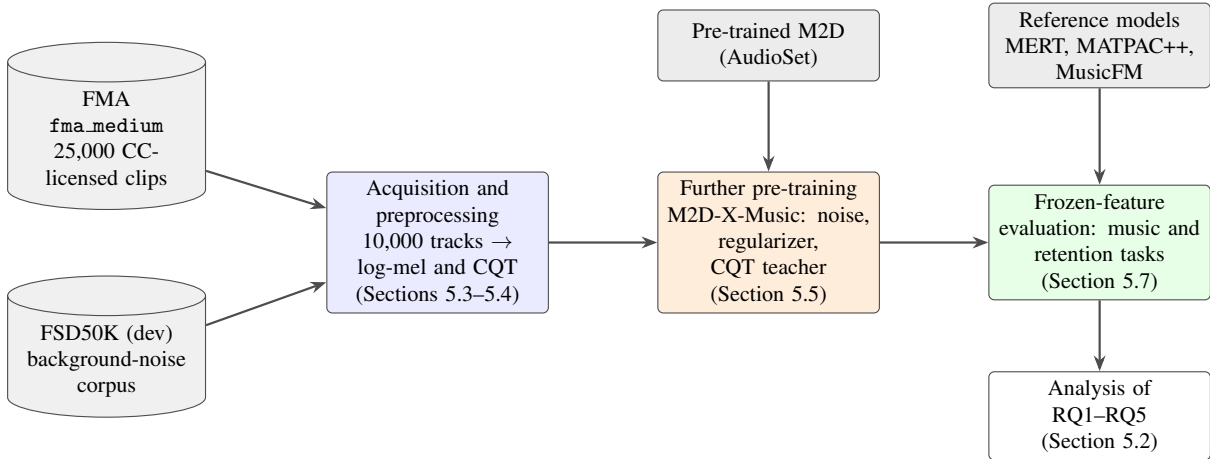


Figure 5.1: Overview of the proposed methodology. A 10,000-track subset of FMA and a background-noise corpus are converted into the input format of the pre-trained M2D model. The model is further pre-trained with the M2D-X-Music objective. The resulting checkpoints and the reference models are then compared on music and retention tasks with frozen features.

5.1 Methodological Foundations

This section reviews the work on which the methodology builds and states, for each source, what is adopted and what is deliberately left out. Table 5.1 at the end of the section collects these decisions.

5.1.1 Masked Latent Prediction and M2D

Masked prediction teaches an encoder to infer hidden parts of an input from the visible parts. Masked autoencoders reconstruct the hidden input values, first for images [7] and then for audio spectrograms [8],

so the learning signal depends partly on low-level detail. Methods that predict targets produced by a separate network instead avoid this dependence: BYOL [9], data2vec [10] and I-JEPA [11] regress the continuous latents of a slowly updated target network, and BEATs [12] predicts discrete labels from an acoustic tokenizer. Masked Modeling Duo (M2D) [13, 14] applies this idea to audio spectrograms with one distinguishing choice: the target network encodes *only the masked patches*, so the target does not carry information about the visible patches that the online network could exploit, and both networks are encouraged to model the input.

The released M2D model [15] takes a log-mel spectrogram of 6 s of audio at 16 kHz with 80 mel bins (80×608 frames) and splits it into 16×16 patches, giving $N_F = 5$ frequency positions and $N_T = 38$ time positions, that is, $N = 190$ patches. A vanilla ViT-Base encoder [16] with 768-dimensional tokens and about 86 M parameters encodes the visible 30 % of the patches. A shallow transformer predictor, taken from the MAE decoder, receives the encoded visible patches together with shared learnable mask tokens and predicts the representations of the masked patches. The target encoder is an exponential moving average (EMA) of the online encoder. The model was pre-trained on AudioSet [17] (2,005,132 clips, 5,569 h) for 300 epochs with an effective batch size of 2,048 [14], and it achieved top-level linear-evaluation results across environmental, speech and music tasks. Related members of the family add an unsupervised classification objective (MATPAC [18]) or multiple prediction hypotheses (MATPAC++ [19]), and M2D-CLAP aligns the representation with text [20]. M2D is chosen as the base model of this study for three practical reasons: the code and the AudioSet weights are public [15], an official recipe for further pre-training exists, and its ViT-Base encoder is small enough for a single 16 GB GPU.

5.1.2 Specializing a General Model: M2D-X

M2D-X extends M2D to learn representations specialized to an application X whose data are scarce and differ from the pre-training distribution [14]. It adds three ingredients. First, background noise from another corpus is mixed into the input of M2D. This acts as data augmentation, which matters because masked prediction normally needs large datasets [21], and, because an additional network sees the clean signal, it forms a denoising task. Second, an *offline network* receives the clean input and provides an additional training signal through a configurable loss, which can be supervised labels, the features of a domain model (distillation), or the features of the original pre-trained model (regularization). Third, the total loss is a weighted sum of the M2D loss and the offline loss. The earlier speech variant, M2D for Speech, distilled HuBERT features while denoising and reached results close to HuBERT on several speech tasks [22, 23].

The evidence most relevant to this project comes from the ICBHI 2017 respiratory-sound task, where

only 539 training recordings (about 5.5 h for the whole database) were available [14]. The AudioSet model (a variant with 16×4 patches and 2-s inputs) scored 61.93 on the task metric. Further pre-training with the M2D loss alone and no background noise *lowered* the score to 61.17, indicating overfitting to the small corpus. Adding background noise (mixing ratio 0.3) raised it to 62.73, and the full M2D-X, which adds the frozen-teacher regularizer, reached 63.29. The training list was replicated to 10,000 samples per epoch, 600 epochs were run with an effective batch size of 128, and the run took about 7 hours on one 24 GB GPU. The authors’ guide [15] states that this procedure suits application datasets of fewer than about 10,000 samples and that an effective batch size of 128 with 600 epochs worked for 10,000 samples. The 10,000-track budget of this project therefore sits at the upper end of the regime the recipe was designed for, which makes it a natural, low-risk starting point.

5.1.3 Music-Specific Self-Supervised Models

MERT. MERT [24] follows HuBERT-style masked prediction on raw audio with two teachers. The *acoustic teacher* supplies discrete targets, obtained either by K-means on log-mel and chroma features or from the residual vector quantization codes of EnCodec [25]. The *musical teacher* is a linear head on the encoder output that regresses the constant-Q transform (CQT) [26] of the audio with a mean-squared-error loss, and the total loss is $\alpha \mathcal{L}_H + \mathcal{L}_{\text{CQT}}$. The CQT has logarithmically spaced bins, so every octave has the same number of bins, which suits pitch and harmony. Adding the CQT term raised key-detection accuracy from 55.1 to 65.0 and GTZAN genre accuracy from 75.2 to 78.6 for the K-means teacher, and from 60.5 to 63.2 and from 72.8 to 77.2 for the RVQ teacher [24]. MERT also mixes short excerpts from the same batch into each clip (in-batch noise mixup) with probability 0.5 in its final setting. Its official implementation computes the CQT magnitude with nnAudio [27] using 84 bins at 12 bins per octave starting at 32.7 Hz [28], which is the configuration adopted here for the musical teacher. The base model was trained on about 1,000 h of music in the paper and on 160,000 h for the 330 M-parameter model [24]; the public MERT-v1-95M checkpoint used in this study was trained on 20,000 h [29].

MusicFM and MuQ. MusicFM [30] adapts the BEST-RQ idea [31] to music: targets are produced by a frozen random projection and a random codebook applied to normalized log-mel features, and a Conformer is trained to predict them. A variant trained on the whole FMA (about 8,000 h) scored below the variant trained on a 160,000-h in-house corpus on most downstream tasks, which the authors attribute to scale and to the crowd-sourced nature of FMA [30]. MuQ [32] predicts Mel-RVQ tokens and reports that it outperforms earlier self-supervised music models with only about 900 h of open-source pre-training data. These two models show that the FMA corpus is usable but not ideal, and that the choice of target matters more than its sophistication; they are used as reference models and their target

definitions are not adopted, because both require pre-training from scratch.

MATPAC and MATPAC++. MATPAC combines masked latent prediction with a DINO-style unsupervised classification objective [33], with total loss $(1 - \alpha)\mathcal{L}_{\text{cls}} + \alpha\mathcal{L}_{\text{pred}}$ and $\alpha = 0.5$ [18]. MATPAC++ replaces the single prediction by several hypotheses selected with annealed multiple choice learning [34, 19]. The finding that matters here is the effect of domain-specific training. When MATPAC++ was trained only on music (about 0.9 M Million Song Dataset samples plus about 0.18 M web-radio samples, roughly 30 h on four H100 GPUs for 6-s segments), its average over eight music tasks rose from 47.2 to 47.9 compared with training on AudioSet, a gain of 0.7 points [19]. In the same table the AudioSet-trained M2D averages 46.3 against 44.2 for MERT, although the MERT scores were taken from its paper and the protocols are not identical. In short, general-audio models are already strong on music tasks, and large-scale music-only training adds little. This is the observation that motivates studying whether a *small, cheap* music-specific step can recover a comparable increment.

5.1.4 Further Pre-training, Forgetting and Regularization

Continuing self-supervised pre-training on domain data is an established idea. In language modeling, domain-adaptive pre-training improves downstream tasks in both high- and low-resource settings [35]. In speech, adding unlabeled in-domain data to pre-training reduces most of the gap caused by domain shift [36]. The risks are equally well documented. Vision transformers further pre-trained on small target data overfit, which motivates distilling from a teacher that was itself further pre-trained [37]. For audio, SONAR continues pre-training BEATs on domain data and compares it with direct continual pre-training that resumes the original objective unchanged [38]. On FMA-derived data of 30,000 to 50,000 segments, direct continuation reduced the frozen-feature GTZAN F1 score from 86.0 to 76.0 and lowered the AudioSet tagging mAP from 34.8 to 14.7, whereas SONAR, which adds distillation-based regularization and data sampling, raised GTZAN to 88.0 and kept the mAP at 34.7 [38]. Another study with M2D found that, for narrow and small respiratory datasets, models pre-trained on diverse general audio beat models trained on the domain alone, and that further pre-training on a combination of general and domain data improved results [39]. The frozen-teacher term of M2D-X is closely related to learning without forgetting [40] and knowledge distillation [41], and it avoids the parameter-importance estimates required by elastic weight consolidation [42]. These studies jointly imply that naive continuation should be treated as a control that may fail, and that retention of general ability must be measured, not assumed.

Table 5.1: Literature findings and the design decisions they lead to.

Source	Finding used	Decision in this study
M2D, M2D-X [13, 14]	Latent prediction of masked patches; public AudioSet weights and further pre-training code; recipe for fewer than about 10,000 samples	Start from the AudioSet checkpoint with 80×608 input, 16 × 16 patches and masking ratio 0.7; keep its loss as the base objective
M2D-X noise and regularizer [14]	Naive further pre-training degraded the score; noise and a frozen original-model teacher recovered and exceeded it	Use background noise ($\eta = 0.3$) and the original M2D as frozen offline teacher; keep naive and partial variants as controls
MERT [24]	A CQT-regression musical teacher improves key detection and genre classification; in-batch mixup helps the RVQ variant	Add a CQT teacher (84 bins) through the offline branch with weight 1; do not train an acoustic tokenizer
MATPAC, MATPAC++ [18, 19]	Music-only pre-training at scale adds about 0.7 points over AudioSet training; public checkpoints	Use their checkpoints as references; do not add the classification head or multiple hypotheses
MusicFM, MuQ [30, 32]	FMA-only training trails a larger corpus; about 900 h of open data is enough for competitive results	Use as references with matched or larger data; do not adopt quantizer targets
Domain-adaptive pre-training, SONAR [35, 38, 37]	Naive continuation can hurt and forget; regularization and replay of general data help	Measure retention on environmental-sound tasks and report a forgetting score
FMA [43]	Creative Commons audio; 30-s clips; artist-filtered train, validation and test splits; curated medium subset	Sample from the training split of the medium subset only; cap tracks per artist; use no genre labels for pre-training
EVAR, MARBLE, HEAR [15, 44, 45, 46]	Frozen-feature linear probing is the common protocol for comparing audio representations	Evaluate every model with the same linear-probe protocol

5.1.5 Design Decisions Derived from the Literature

Table 5.1 summarizes, source by source, which finding is used and which decision follows from it in this study.

5.2 Research Questions and Hypotheses

The study is organized around five questions. Each is paired with a hypothesis that can be contradicted by the planned measurements. The arms and metrics named below are defined in Sections 5.6 and 5.7.

- **RQ1.** Does further pre-training of the AudioSet-pretrained M2D on about 83 h of FMA music improve its frozen-feature performance on music tasks? *H1:* the full recipe (A4) improves the average music score over the unmodified model (A0), whereas naive continuation (A1) does not.
- **RQ2.** Which parts of M2D-X are needed on music data? *H2:* the order of the music score is $A1 < A2 < A3$, that is, background noise and the frozen-teacher regularizer each add a gain, as in the respiratory-sound study.

- **RQ3.** Does a CQT musical teacher add value beyond M2D-X? *H3:* A4 exceeds A3 on pitch-related tasks (NSynth pitch) by more than the probe-seed uncertainty, and is not worse elsewhere.
- **RQ4.** How much of the gap to music-specific models does the cheap step close, and is general-audio ability kept? *H4:* the best arm closes a clearly positive fraction of the gap to the public MERT checkpoints and loses less than 1 point of average environmental-sound accuracy (a tolerance fixed in advance), while a gap to the larger, longer-trained references may remain.
- **RQ5.** How does the benefit depend on the number of FMA tracks? *H5:* the gain increases from 1,000 to 10,000 tracks with diminishing returns, so that most of it is reached before 10,000 tracks.

An outcome contradicting a hypothesis, such as no benefit of the CQT teacher or a collapse of retention, is a valid finding and will be reported as such.

5.3 Dataset Acquisition

This section describes where the pre-training audio comes from, which part of it is eligible, and how the 10,000-sample subset is drawn. Throughout this report a *sample* is one 30-s FMA clip, that is, one track, so 10,000 samples correspond to 10,000 tracks or about 83.3 h of music. This is about 1.5 % of the 5,569 h of AudioSet on which M2D was trained.

5.3.1 Choice of Corpus

The pre-training corpus must satisfy five requirements: it can be obtained and used legally for research; it provides full-quality audio, not only derived features or links; its metadata identify artists and provide predefined splits, so that leakage can be controlled; it is small enough to download and process on one workstation; and it has been used in earlier music self-supervised work, so that results can be compared. The Free Music Archive (FMA) [43] meets all five requirements. Its audio is licensed under Creative Commons licenses chosen by the artists, the tracks are MP3 files, most of them stereo at 44.1 kHz and 320 kbit/s (263 kbit/s on average), and the metadata include artist identifiers and an official split with an artist filter. FMA has been used to pre-train MusicFM [30], it is part of the data of the public MERT-v0-public model according to its model card [29], and FMA-derived data were used for continual pre-training of BEATs [38]. The alternatives are less suitable: the Million Song Dataset does not distribute audio, AudioSet consists of links to online videos whose availability changes, which already makes published results hard to compare [19], and MagnaTagATune (32 kbit/s, 16 kHz mono) and GTZAN (about 1,000 clips with documented faults [47]) are too small or too low in quality for pre-training and are used only as evaluation sets.

5.3.2 Archives and Metadata

FMA is distributed as one metadata archive (`fma_metadata.zip`) and four audio archives that are nested subsets of one another [43, 48]. The metadata table `tracks.csv` describes all 106,574 tracks, with artist, album, genre, license, and the columns that give the subset and the official training, validation and test split of each track. In the medium, small and large subsets the audio of a track is a 30-s clip cut from the middle of the full track, or the whole track if it is shorter. The medium subset contains 25,000 clips with a single top-level genre each, selected according to the completeness of their metadata and their popularity in the hope of selecting tracks of higher quality; the small subset is the balanced selection of 1,000 clips from each of the eight most popular genres of the medium subset [43]. The splits are 80 %/10 %/10 % for training, validation and test, stratified by genre, with an artist filter so that every artist appears in only one split, and a track keeps the same split in every subset [43].

Table 5.2: FMA audio archives and their use in this study. Sizes are those listed by the dataset maintainers [48]; clip counts and genre counts are from the dataset paper [43].

Archive	Clips	Genres	Size (GiB)	Use in this study
<code>fma_small</code>	8,000	8 (balanced)	7.2	Audio contained in the medium archive; official test split used for in-domain genre evaluation
<code>fma_medium</code>	25,000	16 (unbalanced)	22	Source of the pre-training pool (training split only) and of the <code>fma_small</code> evaluation clips
<code>fma_large</code>	106,574	161	93	Not used (larger download, no additional split advantage)
<code>fma_full</code>	106,574	161	879	Not used (untrimmed tracks)

The study downloads the metadata archive and the medium audio archive only, verifies the SHA-1 checksums published with the dataset, and unpacks the files. Because the small subset is contained in the medium subset, the single audio archive also supplies the clips for the in-domain evaluation.

5.3.3 Eligibility Rules and Subset Selection

A track enters the candidate list only if it satisfies all of the following rules.

1. It belongs to the small or medium subset and to the *training* split. Since the official splits are artist-filtered, no artist from the validation or test split contributes to pre-training, which prevents in-domain leakage into the FMA evaluation task. About 80 % of the 25,000 clips, that is, about 20,000 tracks, are expected to qualify; the exact count is recorded when the metadata are read.
2. Its license field is present in the metadata.
3. It passes the audio audit described in Section 5.4.
4. At most $k = 5$ tracks of the same artist are admitted. This limits the influence of prolific artists in

a corpus with a long-tailed artist distribution.

No genre label is used for selection, so the subset keeps the genre distribution of the medium subset, in which experimental music is under-represented [43]; the genre histogram of the chosen subset is reported so that this property is visible. Pre-training itself uses no labels at all.

Algorithm 1 fixes the selection. A seeded random permutation of the eligible tracks defines the order in which candidates are examined. The first 10,000 accepted tracks form the main subset, and shorter prefixes of the same list form the nested subsets with 1,000, 2,500 and 5,000 tracks, plus a 20,000-track extension if time allows. Using prefixes of one list means that all arms and all sizes draw on the same data, so a difference between two results cannot come from a different selection of tracks. Only the ordered list of track identifiers and the seed need to be stored to reproduce the subset.

Data: metadata table T ; seed s ; artist cap k ; sizes $n_1 < n_2 < \dots < n_J$

Result: ordered list L and nested subsets $S_j = L[1..n_j]$; rejection log

$E \leftarrow \{t \in T : t.\text{subset} \in \{\text{small}, \text{medium}\}, t.\text{split} = \text{training}, t.\text{license} \neq \emptyset\}$;

$\pi \leftarrow$ random permutation of E generated with seed s ;

$L \leftarrow$ empty list; $\text{count}[a] \leftarrow 0$ for every artist a ;

foreach track t in π , **in order do**

if $\text{count}[t.\text{artist}] < k$ **and** $\text{Audit}(t)$ passes **then**

 append t to L ;

$\text{count}[t.\text{artist}] \leftarrow \text{count}[t.\text{artist}] + 1$;

end

if $|L| = n_J$ **then** leave the loop;

end

return L , the subsets S_j , and the log of rejected tracks with their reasons;

Algorithm 1: Seeded, artist-capped selection of nested pre-training subsets

5.3.4 Background-Noise Corpus

The denoising component of M2D-X needs a corpus of background sounds. The official recipe uses the FSD50K dataset [49, 14], which contains 51,197 Freesound clips (108.3 h) under Creative Commons licenses, labeled with 200 classes of the AudioSet ontology. Only the development portion is used as noise, and the evaluation portion is left untouched. Because the ontology includes musical instruments, an optional ablation restricts the pool to clips that carry no music-related label, to test whether purely non-musical noise is a better distractor for music (Section 5.6).

5.3.5 Licensing and Ethics

FMA tracks carry artist-chosen Creative Commons licenses that permit research use. Audio and the spectrograms derived from it are kept local and are not redistributed; only the list of track identifiers and the seed are published. The metadata contain only public artist information and no personal data.

The CC-BY-NC license of the MERT weights [29] is respected by using them for non-commercial comparison only.

5.4 Data Preprocessing Pipeline

Figure 5.2 shows the pipeline. Its guiding rule is that the audio must reach the model through exactly the same front-end as in the original pre-training, because a pre-trained encoder that is fed differently computed spectrograms suffers a distribution shift that would be confounded with the effect under study.

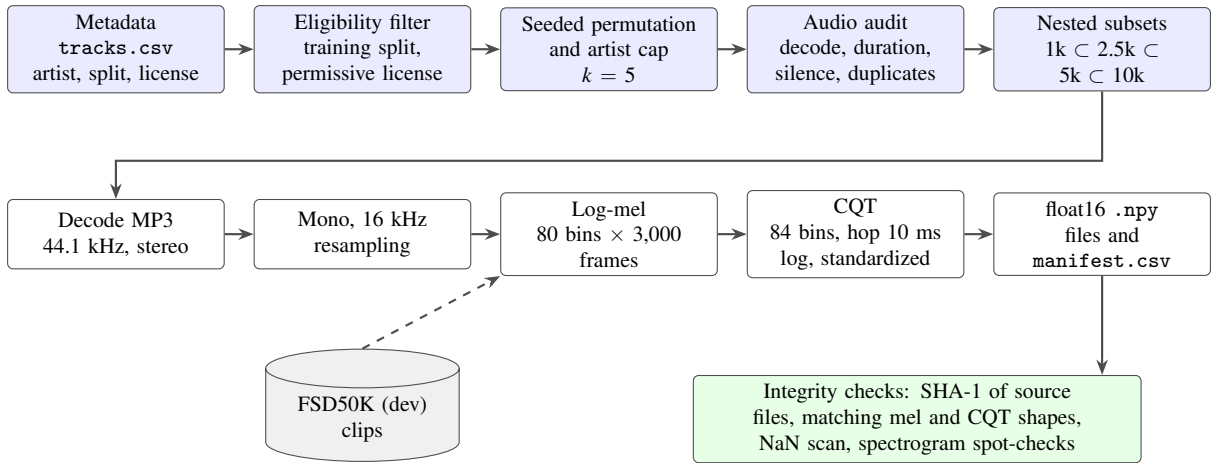


Figure 5.2: Data acquisition and preprocessing pipeline. The top row operates on metadata and audit results and fixes the ordered list of tracks. The bottom row converts the selected files with the same front-end that was used to pre-train M2D. The noise corpus passes through the log-mel stage only (dashed arrow).

5.4.1 Audit and Decoding

Each candidate MP3 file is decoded before it is accepted. The FMA maintainers document corrupted or truncated tracks, tracks shorter than 30 s, and duplicates [48], so the audit applies four rejection rules.

1. **Decoding failure:** the decoder reports an error or a truncated stream.
2. **Duration:** the decoded signal is shorter than 10 s, so that several different 6.08-s windows cannot be cut from it.
3. **Silence:** more than half of the frames have a power more than 60 dB below that of the loudest frame of the clip.
4. **Duplication:** the track has the same normalized pair of artist identifier and title as an accepted track, or the same hash of the decoded 16-kHz waveform. The track with the lowest identifier is kept.

Every rejection is logged with its reason, and the next candidate in the permutation takes the place of a

rejected track (Algorithm 1).

5.4.2 Front-End

The conversion script distributed with the M2D code (`wav_to_lms.py`) [15] is used without modification. It loads each file as a mono signal resampled to 16 kHz and computes a log-mel spectrogram with `nnAudio` [27]. With the mel filter bank H and the short-time Fourier transform X of the waveform, each spectrogram entry is

$$x_{f,t} = \log \left(\sum_k H_{f,k} |X_{k,t}|^2 + \varepsilon \right), \quad f = 1, \dots, 80, \quad (5.1)$$

where ε is a small constant that avoids the logarithm of zero. Table 5.3 lists all parameters. Spectrograms are stored in half precision, which halves the disk space (about 4.8 GB for 10,000 clips of 3,000 frames) at a rounding error far below the standard deviation of the data, and they are cast to single precision when loaded.

Table 5.3: Front-end and target parameters. Values of the log-mel front-end and of the input geometry follow the released M2D model [15, 14]; the CQT settings follow the official MERT implementation [28], with the hop size changed to match the log-mel frame grid.

Item	Value
Audio decoding	Channel mean to mono, resampled to 16 kHz
STFT	Window and FFT size 400 samples (25 ms), hop 160 samples (10 ms)
Mel bands	80 bands between 50 Hz and 8,000 Hz
Input window	608 frames (≈ 6.08 s); patches of 16×16 ; $N_F = 5$, $N_T = 38$, $N = 190$ patches
Normalization	Mean $\mu_{AS} = -7.1$ and standard deviation $\sigma_{AS} = 4.2$ of the AudioSet pre-training data
CQT target	Magnitude, 84 bins at 12 per octave from 32.7 Hz (C1) to about 3.95 kHz, hop 160 samples; logarithm, standardized per bin
Target token rate	Average over $P = 16$ frames (160 ms) to match one patch step

5.4.3 Cropping and Normalization

At every visit, a random window of 608 frames is cut from the stored 30-s spectrogram, as in the official loader. A 30-s track contains about five non-overlapping windows, and random start positions give many more overlapping ones, so the 10,000 tracks supply a large number of distinct inputs even though the track count is small. The window is standardized as

$$\bar{x} = \frac{x - \mu_{AS}}{\sigma_{AS}}, \quad (5.2)$$

with the AudioSet statistics of Table 5.3, so that the pre-trained encoder initially sees inputs of the same scale as during its original pre-training. The official M2D-X code instead recomputes these statistics

from the application data at run time. The deviation is deliberate: it removes one source of change at the start of training. The mean and standard deviation of the selected FMA subset are computed and reported so that the size of the shift is known, and training with the FMA statistics is kept as an optional ablation.

5.4.4 Musical-Teacher Targets

The musical teacher uses the constant-Q transform [26], whose center frequencies are spaced logarithmically,

$$f_k = f_{\min} 2^{(k-1)/12}, \quad k = 1, \dots, 84, \quad f_{\min} = 32.7 \text{ Hz}, \quad (5.3)$$

so that every semitone has its own bin and the top bin lies at about 3.95 kHz, below the Nyquist frequency of 8 kHz. Let $C_{k,t}$ be the CQT magnitude computed with nnAudio on the same 16-kHz mono waveform and with the same hop of 160 samples as the log-mel spectrogram. The target is the log magnitude standardized per bin with the mean m_k and standard deviation s_k over the selected subset, followed by average pooling over the $P = 16$ frames of each token time step τ :

$$\tilde{c}_{k,t} = \frac{\log(C_{k,t} + \varepsilon) - m_k}{s_k}, \quad \bar{c}_{k,\tau} = \frac{1}{P} \sum_{t=(\tau-1)P+1}^{\tau P} \tilde{c}_{k,t}. \quad (5.4)$$

The choices differ from MERT in three respects, each for a reason. MERT regresses the linear magnitude on masked frames at its own frame rate; here the logarithm and the per-bin standardization keep the target on a scale comparable to the log-mel input and avoid a regression dominated by a few loud low-frequency bins, and the pooling matches the 160-ms time step of an M2D token. The CQT is deterministic and needs no additional network, and nnAudio is already a dependency of the M2D front-end. Because the two transforms use the same hop and centered frames, the log-mel and CQT arrays of a clip have the same number of frames; if they differ by a frame because of padding, both are trimmed to the shorter, and one random start index is used to crop both.

5.4.5 Storage, Manifest and Integrity Checks

For each accepted track the pipeline writes one half-precision array for the log-mel spectrogram (about 4.8 GB for 10,000 clips) and one for the CQT (about 5.0 GB), and a row in a manifest file with the track identifier, artist identifier, file paths, frame counts, SHA-1 of the source file, audit outcome and split. The noise corpus is converted to log-mel arrays only. The following checks are run and their outcome is saved: the SHA-1 of every source file matches the published checksum; log-mel and CQT frame counts agree; no array contains NaN or infinite values; value ranges lie within a plausible interval of the normalized scale; and a gallery of 20 randomly chosen spectrograms is inspected by eye against

the audio. The conversion is resumable and driven by one configuration file, so an interrupted run does not repeat finished tracks.

5.5 Further Pre-training Framework

This section defines M2D-X-Music, the objective and the training procedure applied to the AudioSet-pretrained M2D. Figure 5.3 shows the data flow.

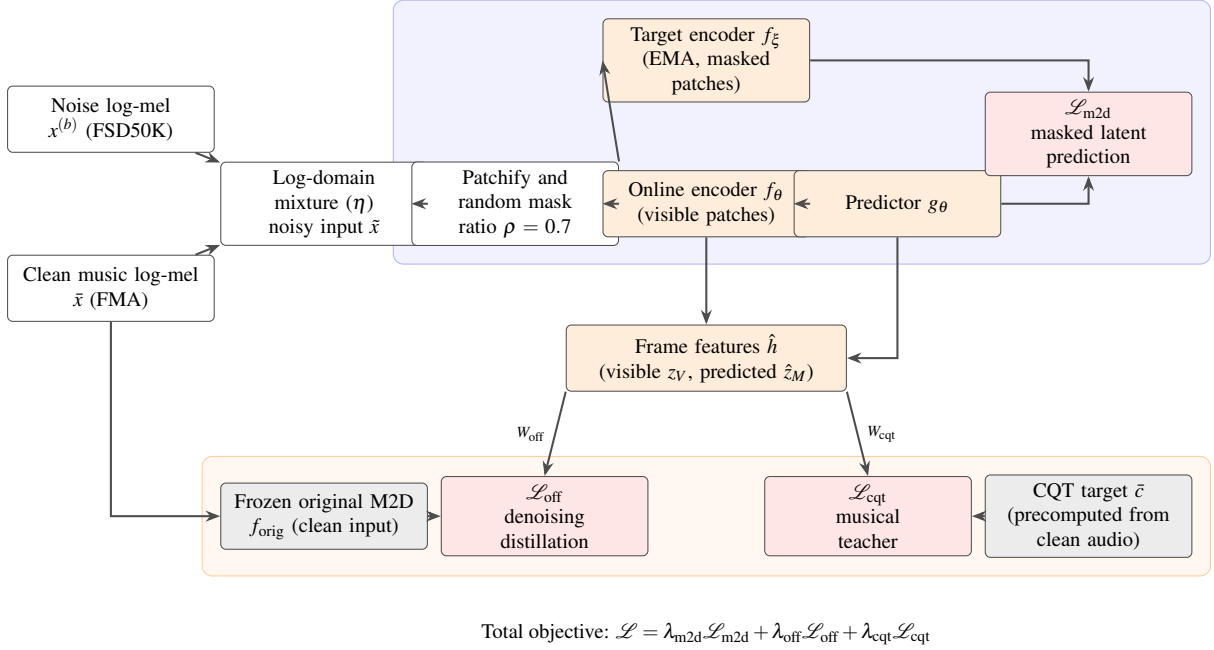


Figure 5.3: Architecture of the further pre-training framework. The online and target networks receive the noisy input (top region), whereas the offline branch receives the clean input and its precomputed targets (bottom region). The frame features $\hat{h}$ are shared by the two offline losses through separate linear mappers W_{off} and W_{cqt} . Only the online encoder f_{θ} is kept after pre-training.

5.5.1 Overview, Initialization and Alternatives Considered

Training starts from the released checkpoint M2D/0.7 (80×608 input, 16×16 patches, masking ratio 0.7) [15]. The online encoder f_{θ} , the predictor g_{θ} and the EMA target encoder f_{ξ} are all initialized from this checkpoint, and a frozen copy f_{orig} of the original encoder serves as offline teacher. Two linear mappers, W_{off} and W_{cqt} , are newly initialized. The online and target networks see the *noisy* input, whereas the offline branch sees the *clean* input and its precomputed targets. After pre-training only the online encoder is kept; its output features are what the evaluation of Section 5.7 uses.

Four alternatives were considered and not adopted. Pre-training on FMA from scratch would answer a different question and is far beyond the single-GPU budget. Restricting updates to adapter or low-rank parameters would reduce forgetting but changes the model the study sets out to compare, and it is better left to later work. Distilling the features of a frozen MERT model is the exact analogue of the

HuBERT distillation of M2D for Speech, but it needs a second, 24-kHz raw-audio branch and a second data format; it is kept as an optional extension. Replacing the objective by quantizer targets in the style of MusicFM or MuQ would require training the whole model again.

5.5.2 Masked Latent Prediction on Noisy Music

The standardized window is cut into $N = N_F N_T = 190$ patches. A random subset $\mathcal{M}$ of $|\mathcal{M}| = \text{round}(\rho N) = 133$ patches is masked with $\rho = 0.7$, and the remaining 57 patches form the visible set $\mathcal{V}$. The online encoder maps the visible patches to $z_{\mathcal{V}} = f_{\theta}(\tilde{x}_{\mathcal{V}})$, and the predictor, given mask tokens m and positional encodings p , predicts all token representations,

$$\hat{z} = g_{\theta}(\text{concat}(z_{\mathcal{V}}, m) + p), \quad (5.5)$$

of which only the masked ones, $\hat{z}_{\mathcal{M}}$, are used in the loss. The target encoder receives the masked patches only, $z_{\mathcal{M}} = f_{\xi}(\tilde{x}_{\mathcal{M}})$, and its output is standardized to zero mean and unit variance, giving $\tilde{z}_{\mathcal{M}}$. The loss is the mean squared error between the ℓ_2 -normalized prediction and target,

$$\mathcal{L}_{\text{m2d}} = \frac{1}{|\mathcal{M}|} \sum_{j \in \mathcal{M}} \left\| \frac{\hat{z}_j}{\|\hat{z}_j\|_2} - \frac{\tilde{z}_j}{\|\tilde{z}_j\|_2} \right\|_2^2 = \frac{1}{|\mathcal{M}|} \sum_{j \in \mathcal{M}} \left(2 - 2 \frac{\langle \hat{z}_j, \tilde{z}_j \rangle}{\|\hat{z}_j\|_2 \|\tilde{z}_j\|_2} \right). \quad (5.6)$$

No gradient flows into the target encoder. Its parameters follow the online encoder by an exponential moving average,

$$\xi \leftarrow \tau_k \xi + (1 - \tau_k) \theta, \quad \tau_k = \tau_0 + (\tau_1 - \tau_0) \frac{k}{K}, \quad (5.7)$$

where k is the optimizer step, K the total number of steps, $\tau_0 = 0.99995$ and $\tau_1 = 0.99999$.

5.5.3 Background Noise and Denoising

To build the noisy input, a window of the noise corpus is mixed with the music window in the power domain, after the mean log level of each corpus has been removed so that the two sources start at comparable levels. With the music log-mel window $x^{(m)}$, the noise window $x^{(b)}$, their corpus means μ_m and μ_b , and the mixing ratio η ,

$$x_{\text{mix}} = \log \left((1 - \eta) \exp(x^{(m)} - \mu_m) + \eta \exp(x^{(b)} - \mu_b) \right), \quad \tilde{x} = \frac{x_{\text{mix}}}{\sigma_{\text{AS}}}, \quad (5.8)$$

which is the mixing rule of M2D-X [14] followed by the same scale normalization as in Equation 5.2. Here μ_m is the AudioSet mean μ_{AS} of Table 5.3 and μ_b is the mean of the noise corpus. The noise windows are drawn without replacement by cycling through a shuffled list of the corpus, and short noise clips are repeated to fill the window. The ratio is fixed at $\eta = 0.3$ in the arms that use noise, the value of

the respiratory-sound recipe [14]. Without a second view, the noise would only be an augmentation; the denoising character arises because the offline losses below are computed against the *clean* signal, so the encoder is asked to produce features that describe the music underneath the noise.

5.5.4 Frozen-Teacher Regularization

Both offline losses act on frame-level features. Let $\hat{Z}$ be the grid of $N_F \times N_T$ token representations in which the visible positions hold the encoder outputs $z_{\mathcal{V}}$ and the masked positions hold the predictions $\hat{z}_{\mathcal{M}}$. The feature of time step τ is the concatenation over the N_F frequency positions,

$$\hat{h}_\tau = \text{concat}(\hat{Z}_{1,\tau}, \dots, \hat{Z}_{N_F,\tau}) \in \mathbb{R}^{N_FD}, \quad N_FD = 5 \times 768 = 3840, \quad (5.9)$$

which is the same layout as the 3,840-dimensional frame features used in the linear evaluation of M2D [15]. The frozen original encoder f_{orig} processes the *clean* window with all patches visible and gives the target features $y_\tau^{\text{off}} \in \mathbb{R}^{3840}$. The regularization loss compares them after a linear map,

$$\mathcal{L}_{\text{off}} = \frac{1}{N_T} \sum_{\tau=1}^{N_T} \left\| \frac{W_{\text{off}} \hat{h}_\tau + b_{\text{off}}}{\|W_{\text{off}} \hat{h}_\tau + b_{\text{off}}\|_2} - \frac{y_\tau^{\text{off}}}{\|y_\tau^{\text{off}}\|_2} \right\|_2^2. \quad (5.10)$$

This term anchors the features of the adapted model to those of the original model on clean inputs, in the manner of learning without forgetting [40], while the first term moves them towards the music distribution. In the speech variant the same mechanism distils a domain model; here the teacher is the model itself, so the term is a regularizer, as in the respiratory-sound experiments [14].

5.5.5 Musical Teacher

The musical teacher adds a second offline task on the same frame features. A linear mapper predicts the pooled CQT target of Equation 5.4,

$$\mathcal{L}_{\text{cqt}} = \frac{1}{N_T K_c} \sum_{\tau=1}^{N_T} \|W_{\text{cqt}} \hat{h}_\tau + b_{\text{cqt}} - \bar{c}_\tau\|_2^2, \quad K_c = 84. \quad (5.11)$$

Because $\hat{h}_\tau$ is built from encoder outputs at visible positions and from predictor outputs at masked positions, the term has two roles at once: it asks the encoder to recover the clean pitch content from noisy visible patches, and it asks the predictor to infer the pitch content of hidden regions. This differs from MERT, where the CQT is regressed on masked positions of a single network, and it keeps the M2D-X convention that offline losses act on all frames. Since the target is standardized, the initial value of the loss is of the order of one, comparable to the other two terms, and the weight $\lambda_{\text{cqt}} = 1$ follows MERT’s finding that a unit weight worked best among 1, 2 and 5 [24].

5.5.6 Joint Objective and Optimization

The full objective is the weighted sum

$$\mathcal{L} = \lambda_{\text{m2d}} \mathcal{L}_{\text{m2d}} + \lambda_{\text{off}} \mathcal{L}_{\text{off}} + \lambda_{\text{cqt}} \mathcal{L}_{\text{cqt}}. \quad (5.12)$$

The experimental arms of Section 5.6 differ only in η , λ_{off} and λ_{cqt} ; all other settings are identical and are listed in Table 5.4. They follow the official M2D-X recipe [15, 14] with AdamW [50]. The learning rate is scaled with the effective batch size B_{eff} by the rule of the code base,

$$\text{lr} = \text{blr} \cdot \frac{B_{\text{eff}}}{256} = 3 \times 10^{-4} \cdot \frac{128}{256} = 1.5 \times 10^{-4}. \quad (5.13)$$

The effective batch size is kept at 128 even though the micro-batch is smaller, because the EMA update is applied once per optimizer step, so the number of steps affects the target network; the guide of the authors stresses that the effective batch size matters [15]. Algorithm 2 gives the procedure.

Table 5.4: Fixed settings of the further pre-training runs. Settings in the upper part follow the released M2D model and the M2D-X recipe; the lower part are the choices made for this study.

Item	Setting
Initial weights	M2D/0.7 (m2d_vit_base-80x608p16x16-221006-mr7): online encoder, predictor and target encoder
Network	ViT-Base encoder, 768-dimensional tokens, about 86 M parameters; MAE-style predictor
Input and masking	80×608 log-mel window, 16 × 16 patches ($N = 190$), masking ratio 0.7 (57 visible, 133 masked)
Optimizer	AdamW with $\beta = (0.9, 0.95)$, weight decay 0.05, gradient-norm clipping at 3.0
Learning rate EMA decay	Base 3×10^{-4} scaled to 1.5×10^{-4} ; 24 warm-up epochs; cosine decay to zero Linear from 0.99995 to 0.99999 over training
Epochs and data per epoch	600 epochs of 10,000 virtual samples, that is, 6.0 M sample views and 46,875 optimizer steps
Batch	Micro-batch 32, gradient accumulation 4, effective batch 128
Noise	FSD50K development clips, $\eta = 0.3$ in arms with noise
Loss weights	$\lambda_{\text{m2d}} = 1$; $\lambda_{\text{off}} \in \{0, 1\}$; $\lambda_{\text{cqt}} \in \{0, 1\}$
Precision and hardware	Mixed precision on one RTX A4000 (16 GB)
Checkpoints	Saved every 100 epochs; epoch 600 is the reported checkpoint

5.5.7 Compute and Memory Budget

One run processes $600 \times 10,000 = 6.0$ million sample views, which is about 1 % of the roughly 600 million views of the original AudioSet pre-training (300 epochs of about two million clips). The respiratory-sound run of the authors, which had the same number of views, took about 7 hours on one RTX 3090 Ti with 24 GB, with 2-s inputs and 16×4 patches [14]. The RTX A4000 has less memory and lower

Data: initial weights (θ_0, ξ_0) ; frozen teacher f_{orig} ; music list $\mathcal{D}$ with log-mel and CQT arrays; noise list $\mathcal{B}$; η , λ_{m2d} , λ_{off} , λ_{cqt} ; epochs E ; epoch size n ; micro-batch b ; accumulation a

Result: further pre-trained encoder f_θ

$\theta \leftarrow \theta_0$; $\xi \leftarrow \xi_0$; initialize W_{off} , W_{cqt} ;

repeat $\mathcal{D}$ until the epoch list $\mathcal{D}_n$ has n entries;

for $e = 1$ **to** E **do**

 shuffle $\mathcal{D}_n$;

foreach *micro-batch of b clips* **do**

foreach *clip i in the micro-batch* **do**

 draw one start index; crop 608 frames of the log-mel array and of the CQT array at that index;

 take the next crop from the shuffled noise list $\mathcal{B}$ and form $\tilde{x}_i$ with Equation 5.8;

end

 draw random masks; compute $\mathcal{L}_{\text{m2d}}$ with Equations 5.5 and 5.6;

 build $\hat{h}$ (Equation 5.9); compute y^{off} from the clean window with f_{orig} without gradient;

 compute $\mathcal{L}_{\text{off}}$ and $\mathcal{L}_{\text{cqt}}$ (Equations 5.10 and 5.11); form $\mathcal{L}$ (Equation 5.12);

 backpropagate $\mathcal{L}/a$ into θ , W_{off} , W_{cqt} ;

if *a micro-batches have been accumulated* **then**

 clip the gradient norm; apply an AdamW step with the scheduled learning rate;

 update ξ with Equation 5.7;

end

end

if e is a multiple of 100 **then** save a checkpoint of f_θ ;

end

return f_θ ;

Algorithm 2: Further pre-training with the M2D-X-Music objective

throughput, so the budget of this study is an estimate of about 10 to 16 hours per run, to be confirmed by a pilot of a few epochs before the main runs. The micro-batch of 32 is the setting that the authors give for smaller GPUs [15]. If it does not fit into 16 GB, the micro-batch is halved and the accumulation doubled, which leaves the effective batch size of 128 unchanged. If the throughput is lower than expected, the number of epochs is reduced for *all* arms alike, since the authors' training curves show gains emerging after a few hundred epochs [14]; intermediate checkpoints every 100 epochs make this decision visible in the data.

5.6 Experimental Design

The experiments fall into three groups: reference models that are only evaluated, a small set of further pre-training *arms* that differ in one factor at a time, and runs that vary the number of FMA tracks. All pre-training runs are compute-matched, that is, each processes the same number of sample views.

5.6.1 Reference Models

Table 5.5 lists the models against which the further pre-trained checkpoints are compared. They are chosen to separate the effect of data volume from the effect of the objective. MERT-v1-95M and MERT-v0-public have a parameter count similar to that of the M2D encoder (95 M against 86 M) but were trained from scratch on about 20,000 h and about 900 h of music, which is roughly 240 and 11 times the 83 h of FMA used here, and MusicFM-FMA shows what training from scratch on the whole of FMA achieves. Reference models are never fine-tuned or further trained. For transformer references whose layers differ in quality, features of all layers are extracted and the best layer is chosen per task on the validation split; this is a favorable setting for the references and therefore a conservative one for the claims of this study. For the M2D family the final-layer features with the 3,840-dimensional frame layout of the original linear evaluation are used. Audio is resampled to the rate of each model and cut into windows of the context length of the model (5 s for MERT), and the segment embeddings are averaged over time.

Table 5.5: Reference models. Parameter counts and pre-training data are as reported in the cited sources; MATPAC++ values are from its preprint and the checkpoints are the ones published by its authors.

Model	Params	Pre-training data	Role
M2D/0.7 [14]	86 M	AudioSet, 5,569 h	Arm A0: starting point without FMA training
MERT-v1-95M [24, 29]	95 M	About 20,000 h of music; 24 kHz	Music-specific, same size, large data
MERT-v0-public [29]	95 M	About 900 h of open music (Music4All and part of FMA); 16 kHz	Music-specific, same size, small data
MusicFM (FMA) [30, 51]	About 330 M	FMA, about 8,000 h	Music-specific, same corpus family, larger model
MATPAC++ (music) [19, 52]	About 86 M	Million Song Dataset samples and web-radio samples, about 1.1 M in total	Music-specific model of the masked-latent-prediction family
MATPAC++ (general) [19, 52]	About 86 M	AudioSet	General counterpart of the previous row

5.6.2 Pre-training Arms

Table 5.6 defines the arms. The five arms A1 to A5 form a design in which each component is switched on or off, so that every claim about a component rests on a specific, pre-declared comparison rather than on a free search through results:

- A1 against A0: the effect of naive continuation of M2D on FMA (RQ1, RQ2).
- A2 against A1: the effect of background noise.

- A3 against A2: the effect of the frozen-teacher regularizer.
- A4 against A3: the effect of the musical teacher on top of M2D-X (RQ3).
- A5 against A2 and A4 against A5: the effect of the musical teacher without the regularizer, and of the regularizer when the teacher is present.
- S1 to S3 against the best of A3 and A4: the effect of the number of tracks (RQ5).

The best of A3 and A4, called A^* , is chosen by the mean music score on the *validation* splits only, so that the test splits take no part in any choice. If time is short, the arms are run in the order A3, A1, A4, A2, A5, then S1 to S3, then the optional runs.

Table 5.6: Pre-training arms. Dashes mark runs without FMA pre-training. All arms use the same track list, the same random seed for data order and masks, and 6.0 M sample views.

Arm	η	λ_{off}	λ_{cqt}	Tracks	Purpose
A0	—	—	—	—	Unmodified M2D/0.7 (reference point)
A1	0	0	0	10,000	Naive continuation with the M2D loss only
A2	0.3	0	0	10,000	Adds background noise
A3	0.3	1	0	10,000	M2D-X: noise and frozen-teacher regularizer
A4	0.3	1	1	10,000	M2D-X-Music: adds the CQT musical teacher
A5	0.3	0	1	10,000	Musical teacher without the regularizer
S1–S3	0.3	as A^*	as A^*	1,000; 2,500; 5,000	Data-size study with the best arm A^*

Optional runs, executed only if time remains, are: a noise pool without music-related FSD50K classes; a lower noise ratio $\eta = 0.1$; training with the FMA normalization statistics; a 20,000-track run with 300 epochs so that the number of sample views stays at 6.0 M; a second pre-training seed for A3 and A4; and distillation of frozen MERT features as a second domain teacher.

5.6.3 Controls and Reproducibility

Several measures keep the comparison honest. All arms share the track list of Algorithm 1 and the same seed for data order and masking. The reported checkpoint is the one at epoch 600, fixed before any test result is seen; checkpoints at epochs 100 to 500 are evaluated on the validation splits of the small tasks only (GTZAN, FMA-small, ESC-50, UrbanSound8K) to show the learning curve and to detect degradation or collapse, as in the checkpoint analysis of the authors [14]. The official training script also logs label-free feature statistics at fixed intervals, which provides an additional collapse indicator. The seeds, the configuration files, the track list and the manifest are stored with the code. If time permits, a second pre-training seed for A3 and A4 estimates the variance that comes from pre-training itself rather than from the probes.

5.7 Evaluation Protocol

Evaluation uses frozen features and a linear classifier, which isolates the quality of the representation from the capacity of the classifier and is the protocol used for the M2D family, MATPAC and the HEAR and MARBLE benchmarks [14, 18, 46, 45].

5.7.1 Downstream Tasks

Table 5.7 lists the tasks. The music tasks cover genre (GTZAN and FMA-small), instrument family (NSynth), pitch (NSynth) and tagging (MagnaTagATune). The pitch task is included because the CQT teacher injects exactly this kind of information, so it is where the benefit of the teacher should be most visible, as key and pitch detection were in MERT [24]. FMA-small is the only in-domain task; its test clips come from artists who are absent from pre-training because of the artist-filtered split. Two environmental-sound tasks, ESC-50 and UrbanSound8K, measure whether general-audio ability is retained. GTZAN is used with its fault-filtered split [53, 47]. The tasks GTZAN, NSynth (instrument and pitch), ESC-50 and UrbanSound8K are available in the EVAR toolkit of the M2D authors [44]; FMA-small and MagnaTagATune are added by writing task metadata in the same format, using the official FMA split and one fixed published split of MagnaTagATune that is recorded before any evaluation.

Table 5.7: Downstream tasks. Splits are given as training, validation and test sizes where a fixed split exists. The GTZAN and NSynth figures are those used in the M2D evaluation [14]; the FMA-small split follows the dataset paper [43].

Task	Role	Labels	Metric	Split
GTZAN [54]	Music, genre	10 genres	Accuracy	Fault-filtered: 443 / 197 / 290
NSynth, instrument [55]	Music, timbre	11 instrument families	Accuracy	289,205 / 12,678 / 4,096
NSynth, pitch [55]	Music, pitch	MIDI pitch	Accuracy	Same notes as above
FMA-small genre [43]	Music, in-domain	8 genres	Accuracy	Official, artist-filtered: 6,400 / 800 / 800
MagnaTagATune [56]	Music, tagging	50 tags	mAP	One fixed published split
ESC-50 [57]	Retention	50 sound classes	Accuracy	5-fold cross-validation
UrbanSound8K [58]	Retention	10 sound classes	Accuracy	10-fold cross-validation

5.7.2 Frozen-Feature Linear Evaluation

For every task, clips longer than the 6.08-s input of M2D are cut into consecutive windows, each window is encoded with all patches visible, and the frame features of the windows are concatenated along time, which is the first of the two options for variable-length audio described by the authors [14]. The clip

embedding is the temporal mean of the T' frame features h_τ of the encoder,

$$e = \frac{1}{T'} \sum_{\tau=1}^{T'} h_\tau \in \mathbb{R}^{3840}. \quad (5.14)$$

A single linear layer is trained on the standardized embeddings with the Adam optimizer, a learning rate of 3×10^{-5} for at most 200 epochs, and early stopping with a patience of 20 epochs on the validation split, following the evaluation of the M2D authors [14], and the model of the best validation epoch is evaluated on the test split. The softmax cross-entropy loss is used for single-label tasks and the sigmoid binary cross-entropy loss for tagging. Each probe is trained with six random seeds. Because the encoder is frozen, the embeddings of each model are computed once per task and cached, so the cost of evaluating a checkpoint is dominated by the forward pass over the data, and the large NSynth task is evaluated only for the final checkpoints.

5.7.3 Metrics, Aggregates and Statistics

Accuracy is used for single-label tasks and mean average precision (mAP) for tagging, all expressed in percent. Two aggregate scores summarize a model m over the music tasks $\mathcal{T}_{\text{mus}}$ and the retention tasks $\mathcal{T}_{\text{gen}}$, with $s_u(m)$ the score on task u :

$$S_{\text{mus}}(m) = \frac{1}{|\mathcal{T}_{\text{mus}}|} \sum_{u \in \mathcal{T}_{\text{mus}}} s_u(m), \quad S_{\text{gen}}(m) = \frac{1}{|\mathcal{T}_{\text{gen}}|} \sum_{u \in \mathcal{T}_{\text{gen}}} s_u(m), \quad (5.15)$$

and the forgetting score is $\Delta_{\text{gen}}(m) = S_{\text{gen}}(m) - S_{\text{gen}}(\text{A0})$, which is negative when general ability is lost. To express how far the cheap step goes towards a music-specific reference r , the gap-closure ratio on task u is

$$\text{GCR}_u(m; r) = \frac{s_u(m) - s_u(\text{A0})}{s_u(r) - s_u(\text{A0})}, \quad (5.16)$$

where a value of 0 means no change from the unmodified model, 1 means parity with the reference, and values above 1 mean that the adapted model surpasses it. The ratio is unstable when the reference is close to A0, so it is computed only on tasks where $s_u(r) - s_u(\text{A0}) \geq 1$ point, and the number of such tasks is reported next to the mean. In addition, the number of tasks on which a model is within the confidence interval of a reference, or above it, is reported as a parity count.

Uncertainty is handled at two levels. For one trained model, the mean over the $n = 6$ probe seeds is reported with the 95 % interval

$$\bar{s} \pm t_{n-1, 0.975} \frac{\text{sd}}{\sqrt{n}}. \quad (5.17)$$

For the difference between two models on a fixed test split, a paired bootstrap over the test clips with

10,000 resamples gives a 95 % percentile interval of the difference [59]; for the cross-validated tasks the per-fold differences are used instead. A difference between two arms is called *supported* only if the interval of the difference excludes zero and, where a second pre-training seed exists, the sign is the same for both seeds; otherwise it is reported as inconclusive. Only the contrasts listed in Section 5.6 are tested, so that the number of comparisons stays small and fixed in advance.

5.7.4 Threats to Validity

- **Small test sets.** The GTZAN test split has 290 clips, so one clip is worth about 0.34 points and its confidence intervals are several points wide. Conclusions will not rest on GTZAN alone, and the bootstrap intervals are reported.
- **Uncontrolled references.** The reference models differ in size, training data, sampling rate and context length (Table 5.5). Comparisons with them are indicative of what is possible, not controlled experiments; only A0 to A5 differ in one factor at a time.
- **In-domain advantage.** FMA-small shares its source with the pre-training data. Its result is therefore reported separately from the out-of-domain music tasks, and conclusions about general music understanding rely on the latter.
- **Limited tuning and seeds.** The noise ratio and loss weights are taken from the literature and not tuned, the best arm is selected on validation data only, and only one or two pre-training seeds are affordable. Differences smaller than the seed-to-seed gap will not be claimed.
- **Possible overlap with external data.** The AudioSet-pretrained starting point may already have seen music similar to the evaluation sets, and no audio fingerprinting is run between FMA and the external benchmarks, so a small undetected overlap cannot be excluded.
- **Computational nondeterminism.** Seeds are fixed, but GPU kernels are not bitwise deterministic, so repeating a run may give slightly different numbers.

5.8 Conclusion

This chapter presented the proposed approach. A resumable pipeline selects 10,000 tracks of the Free Music Archive from the training split of the medium subset with a seeded, artist-capped, nested sampling procedure, audits them, and converts them with the front-end of the original M2D model into log-mel arrays and into CQT targets for a musical teacher. The further pre-training framework M2D-X-Music extends the M2D-X recipe: the online and target networks receive music mixed with background noise, a frozen copy of the original model regularizes the features on clean input, and an additional linear head

regresses the CQT of the clean audio in the manner of the musical teacher of MERT. The experimental design turns these components on and off in a small number of compute-matched arms, adds runs with fewer tracks, and compares the results with public music-specific models. The evaluation uses a frozen-feature linear protocol on music tasks and on two retention tasks, with aggregate scores, a gap-closure ratio, bootstrap intervals and a pre-declared rule for calling a difference supported. The limits of the design, in particular the small test sets, the uncontrolled references and the single GPU, are stated so that the results can be interpreted correctly. The next chapter presents the high-level and low-level design of the system that implements this approach.

Chapter 6 High-Level and Low-Level Design

Provide the high- and low-level design of your system in this chapter. Select the design that is appropriate for your project.

NOTE: This Chapter is compulsory for both R&D projects and Development projects.

6.1 System Overview

Provide a general description of the software system, including its functionality and matters related to the overall system and its design (perhaps including a discussion of the basic design approach or organization). Feel free to split this discussion up into subsections (and sub-subsections, etc. ...).

6.2 Design Considerations

This section describes many issues that need to be addressed or resolved before attempting to devise a complete design solution.

6.2.1 Assumptions and Dependencies

Describe any assumptions or dependencies regarding the software and its use. These may concern such issues as:

- Related software or hardware
- Operating systems
- End-user characteristics
- Possible and/or probable changes in functionality

Do not just add random generic statements, be clear and concise about what you are writing.

6.2.2 General Constraints

Describe any global limitations or constraints that have a significant impact on the design of the system's software (and describe the associated impact). Such constraints may be imposed by any of the following (the list is not exhaustive):

- Hardware or software environment
- End-user environment
- Availability or volatility of resources

- Standards compliance
- Interoperability requirements
- Interface/protocol requirements
- Data repository and distribution requirements
- Security requirements (or other such regulations)
- Memory and other capacity limitations
- Performance requirements
- Network communications
- Verification and validation requirements (testing)
- Other means of addressing quality goals
- Other requirements described in the requirements specification

6.2.3 Goals and Guidelines

Describe any goals, guidelines, principles, or priorities which dominate or embody the design of the system's software. Such goals might be:

- The KISS principle ("Keep it simple stupid!")
- Emphasis on speed versus memory use
- Working, looking, or "feeling" like an existing product

For each such goal or guideline, unless it is implicitly obvious, describe the reason for its desirability. Feel free to state and describe each goal in its own sub subsection if you wish.

6.2.4 Development Methods

Briefly describe the method or approach used for this software design. If one or more formal/published methods were adopted or adapted, then include a reference to a more detailed description of these methods. If several methods were seriously considered, then each such method should be mentioned, along with a brief explanation of why all or part of it was used or not used.

6.3 System Architecture

This section should describe both the internal architecture of the modules, as well as the external architecture of the system with other systems, if any. Diagrammatic architecture is compulsory.

This section should provide a high-level overview of how the functionality and responsibilities of the system were partitioned and then assigned to subsystems or components. Do not go into too much detail about the individual components themselves (there is a subsequent section for detailed component descriptions). The main purpose here is to gain a general understanding of how and why the system was decomposed, and how the individual parts work together to provide the desired functionality.

At the top-most level, describe the major responsibilities that the software must undertake and the various roles that the system (or portions of the system) must play. Describe how the system was broken down into its components/subsystems (identifying each top-level component/subsystem and the roles/responsibilities assigned to it). Describe how the higher-level components collaborate with each other in order to achieve the required results. Don't forget to provide some sort of rationale for choosing this particular decomposition of the system (perhaps discussing other proposed decompositions and why they were rejected). Feel free to make use of design patterns, either in describing parts of the architecture (in pattern format), or for referring to elements of the architecture that employ them.

If there are any diagrams, models, flowcharts, documented scenarios or use-cases of the system behavior and/or structure, they may be included here (unless you feel they are complex enough to merit being placed in the Detailed System Design section). Diagrams that describe a particular component or subsystem should be included within the particular subsection that describes that component or subsystem.

Note: This section (and its subsections) really applies only to newly developed (or yet-to-be developed) portions of the system. If there are parts of the system that already existed before this development effort began, then you only need to describe the pre-existing parts that the new parts of the system depend upon, and only in enough detail sufficient to describe the relationships and interactions between the old parts and the new parts. Pre-existing parts that are modified or enhanced need to be described only to the extent that is necessary for the reader to gain a sufficient understanding of the nature of the changes that were made.

6.3.1 Subsystem Architecture

If a particular component is one which merits a more detailed discussion than what was presented in the System Architecture section, provide that more detailed discussion in a subsection of the System Architecture section (or it may even be more appropriate to describe the component in its own design document). If necessary, describe how the component was further divided into subcomponents, and

the relationships and interactions between the subcomponents (similar to what was done for top-level components in the System Architecture section).

If any subcomponents are also deemed to further merit discussion, then describe them in a separate subsection of this section (and in a similar fashion). Proceed to go into as many levels/subsections of discussion as needed in order for the reader to gain a high-level understanding of the entire system or subsystem (but remember to leave the gory details for the Detailed System Design section).

If this component is very large and/or complex, you may want to consider documenting its design in a separate document and simply including a reference to it in this section. If this is the option you choose, the design document for this component should have an organizational format that is very similar (if not identical to) this document.

6.4 Architectural Strategies

Explain all strategies that you use for designing of the Architecture. Describe any design decisions and/or strategies that affect the overall organization of the system and its higher-level structures. These strategies should provide insight into the key abstractions and mechanisms used in the system architecture. Describe the reasoning employed for each decision and/or strategy (possibly referring to previously stated design goals and principles) and how any design goals or priorities were balanced or traded-off. Such decisions might concern (but are not limited to) things like the following:

- Use of a particular type of product (programming language, database, library, etc. ...)
- Reuse of existing software components to implement various parts/features of the system
- Future plans for extending or enhancing the software
- User interface paradigms (or system input and output models)
- Hardware and/or software interface paradigms
- Error detection and recovery
- Memory management policies
- External databases and/or data storage management and persistence
- Distributed data or control over a network
- Generalized approaches to control
- Concurrency and synchronization
- Communication mechanisms

- Management of other resources

6.4.1 Architectural Strategies Strategy-1 name or description

6.4.2 Architectural Strategies Strategy-2 name or description

Each significant strategy employed should probably be discussed in its subsection or (if it is complex enough) in a separate design document (with an appropriate reference here, of course). Make sure that when describing a design decision you also discuss any other significant alternatives that were considered and your reasons for rejecting them (as well as your reasons for accepting the alternative you finally chose). Sometimes it will be most effective to employ the "pattern format" for describing a strategy.

6.5 Domain Model/Class Diagram

In this subsection, add the Class Diagram of your system. Class diagrams represent the structure design of your system.

6.6 Policies and Tactics

Describe any design policies and/or tactics that do not have sweeping architectural implications (meaning they would not significantly affect the overall organization of the system and its high-level structures) but which nonetheless affect the details of the interface and/or implementation of various aspects of the system. Such decisions might concern (but are not limited to) things like the following:

- Choice of which specific product to use (compiler, interpreter, database, library, etc. ...)
- Engineering trade-offs
- Coding guidelines and conventions
- The protocol of one or more subsystems, modules, or subroutines
- The choice of a particular algorithm or programming idiom (design pattern) to implement portions of the system's functionality
- Plans for ensuring requirements traceability
- Plans for testing the software
- Plans for maintaining the software
- Interfaces for end-users, software, hardware, and communications

- Hierarchical organization of the source code into its physical components (files and directories).
- How to build and/or generate the system's deliverables (how to compile, link, load, etc. ...)

6.6.1 Policy/tactic-1 name or description

6.6.2 Policy/tactic-2 name or description

Each particular policy or set of tactics employed should probably be discussed in its own subsection, or (if it is large or complex enough) in a separate design document (with an appropriate reference here of course). Make sure that when describing a design decision that you also discuss any other significant alternatives that were considered, and your reasons for rejecting them (as well as your reasons for accepting the alternative you finally chose). For this reason, it may frequently be convenient to use one of the more popular "pattern formats" to describe a given tactic.

For this particular section, it may become difficult to decide whether a particular policy or set of tactics should be discussed in this section, or in the System Architecture section, or in the Detailed System Design section for the appropriate component. You will have to use your own "best" judgment to decide this. There will usually be some global policies and tactics that should be discussed here. Still, decisions about interfaces, algorithms, and/or data structures might be more appropriately discussed in the same (sub) section as its corresponding software component in one of these other sections.

Chapter 7 Implementation and Test Cases

For each chapter, provide a paragraph of introduction and at the end, a paragraph of conclusions. (Not the programming code but the algorithmic and procedural details especially related to the hidden/ backend algorithms that are not covered in the design)

7.1 Implementation

Whatever implementation that you have done so far, please elaborate here. Give clear details of the algorithms that were implemented along with the platform and the APIs which were used. **NOTE: For FYP-1, this chapter can be changed to description of prototype developed.**

7.1.1 Implementation of First Component/Algorithm

Write the implementation of the first component of your system here.

7.2 Test case Design and description

This section will be added in FYP-II. Summarize the common attributes of test cases. This may include input constraints that must be true for every input in the set of associated test cases, any shared environmental needs, any shared special procedural requirements, and any shared case dependencies. The following scheme is recommended for describing test cases in detail. The Sample test case is provided in table 7.1.

Table 7.1: Sample Test case No.1

<Software Component Name>			
<Reference>			
Test Case ID:	Reference Number	QA Test Engineer:	Name of personnel
Test case Version:	Version number	Reviewed By:	Testing Team lead
Test Date:	Date	Use Case Reference(s):	Relation to use cases
Revision History:	Refer to previous test case identity (if any)		
Objective:	Need and scope of the testing		
Product/Ver/Module:	Refer to overall system being built and the place of this test case in it.		
Environment:	Necessary and desired properties of the test environment.		
Assumptions:	Assumptions that might affect the testing process.		
Pre-Requisite:	Necessary condition that needs to be fulfilled before the test case.		
Step No.	Execution description	Procedure result	
	Events being tested.	Mention software response.	
Comments:			
Passed Failed Not Executed			

7.3 Test Metrics

Summarize here the common ground of attributes of test case metrics as explained in Table 7.2.

Table 7.2: Sample Test case Metric.No.1

Metric	Purpose
Number of Test Cases	Total number of test cases that you have developed for your system.
Number of Test Cases Passed	The number of test cases that successfully passed
Number of Test Cases Failed	The number of test cases that failed
Test Case Defect Density	(No of test cases failed * 100) No of test cases executed
Test Case Effectiveness	No of defects detected using test cases *100 Total number of defects detected
Traceability Matrix	Traceability is the ability to determine that each the feature has a source in requirements and each requirement has a corresponding implemented feature.

7.3.1 Sample Test case Metric.No.2

Similar to the above table, add here for test metrics 2,3,4.

Chapter 8 User Manual

This chapter will be added in FYP-II. Design a complete user manual of your application and add that in this chapter. **NOTE: This Chapter is optional for Research projects but compulsory for Development projects.**

Chapter 9 Experimental Results and Discussion

This chapter will be added in FYP-II. Give proper analysis and discussion of experimental results (in plain English text) along with tables of results. For each chapter provide a paragraph of introduction and in the end a paragraph of conclusions.

NOTE: This Chapter is optional for Development projects but compulsory for Research projects.

Chapter 10 Conclusions

10.1 aa

10.1.1 dfs

10.1.1.1 dsd

This chapter is mandatory. Give conclusions and summary of the work done. What were your findings and what were the results? Discuss in detail whether the scope of your project was entirely covered or not and whether the objectives of the project were met or not. What challenges did you face and what has been left out and why.

Sum up all the conclusions of all the chapters here to make a final conclusion chapter. Do not repeat any text, just summarize it in different words. Give recommendations for future work also.

For FYP-1 it is mandatory to list down a plan of the work to be done for FYP-2.

Bibliography

- [1] I. Guyon, A. Saffari, G. Dror, and G. Cawley, “Analysis of the IJCNN 2007 agnostic learning vs. prior knowledge challenge,” *Neural Networks*, vol. 21, pp. 544–550, December 2008.
- [2] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [3] R. O. Duda, P. E. Hart, and D. G. Stork, *Pattern Classification*. John Wiley and Sons, 2000.
- [4] C.-C. Chang and C.-J. Lin, *LIBSVM: A Library for Support Vector Machines*, 2001. Available: <http://www.csie.ntu.edu.tw/~cjlin/libsvm>.
- [5] I. Guyon, A. Saffari, G. Dror, and G. Cawley, “Agnostic learning vs. prior knowledge challenge,” in *Proceedings of International Joint Conference on Neural Networks*, August 2007.
- [6] “Agnostic Learning Vs. Prior Knowledge Challenge by International Joint Conference on Neural Networks (IJCNN).” Available: <http://www.agnostic.inf.ethz.ch> [Accessed November 02, 2018].
- [7] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, “Masked autoencoders are scalable vision learners,” in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 16000–16009, 2022.
- [8] P.-Y. Huang, H. Xu, J. Li, A. Baevski, M. Auli, W. Galuba, F. Metze, and C. Feichtenhofer, “Masked autoencoders that listen,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.
- [9] J.-B. Grill, F. Strub, F. Altché, C. Tallec, P. H. Richemond, E. Buchatskaya, C. Doersch, B. A. Pires, Z. D. Guo, M. G. Azar, B. Piot, K. Kavukcuoglu, R. Munos, and M. Valko, “Bootstrap your own latent: A new approach to self-supervised learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.
- [10] A. Baevski, W.-N. Hsu, Q. Xu, A. Babu, J. Gu, and M. Auli, “data2vec: A general framework

- for self-supervised learning in speech, vision and language,” in *Proc. International Conference on Machine Learning (ICML), PMLR*, vol. 162, pp. 1298–1312, 2022.
- [11] M. Assran, Q. Duval, I. Misra, P. Bojanowski, P. Vincent, M. Rabbat, Y. LeCun, and N. Ballas, “Self-supervised learning from images with a joint-embedding predictive architecture,” in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 15619–15629, 2023.
- [12] S. Chen, Y. Wu, C. Wang, S. Liu, D. Tompkins, Z. Chen, W. Che, X. Yu, and F. Wei, “BEATs: Audio pre-training with acoustic tokenizers,” in *Proc. International Conference on Machine Learning (ICML), PMLR*, vol. 202, pp. 5178–5193, 2023.
- [13] D. Niizumi, D. Takeuchi, Y. Ohishi, N. Harada, and K. Kashino, “Masked Modeling Duo: Learning representations by encouraging both networks to model the input,” in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 1–5, 2023.
- [14] D. Niizumi, D. Takeuchi, Y. Ohishi, N. Harada, and K. Kashino, “Masked Modeling Duo: Towards a universal audio pre-training framework,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 32, pp. 2391–2406, 2024.
- [15] NTT Communication Science Laboratories, “M2D: Masked modeling duo, official implementation and pre-trained weights.” <https://github.com/nttcsllab/m2d>. Accessed: October 2026.
- [16] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, “An image is worth 16x16 words: Transformers for image recognition at scale,” in *Proc. International Conference on Learning Representations (ICLR)*, 2021.
- [17] J. F. Gemmeke, D. P. W. Ellis, D. Freedman, A. Jansen, W. Lawrence, R. C. Moore, M. Plakal, and M. Ritter, “Audio Set: An ontology and human-labeled dataset for audio events,” in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 776–780, 2017.
- [18] A. Quenennec, P. Chouteau, G. Peeters, and S. Essid, “Masked latent prediction and classification for self-supervised audio representation learning,” in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 1–5, 2025.
- [19] A. Quenennec, P. Chouteau, G. Peeters, and S. Essid, “MATPAC++: Enhanced masked latent prediction for self-supervised audio representation learning.” arXiv preprint arXiv:2508.12709, 2025. Preprint, under review.

- [20] D. Niizumi, D. Takeuchi, Y. Ohishi, N. Harada, M. Yasuda, S. Tsubaki, and K. Imoto, “M2D-CLAP: Masked Modeling Duo meets CLAP for learning general-purpose audio-language representation,” in *Proc. Interspeech*, pp. 57–61, 2024.
- [21] Z. Xie, Z. Zhang, Y. Cao, Y. Lin, Y. Wei, Q. Dai, and H. Hu, “On data scaling in masked image modeling,” in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 10365–10374, 2023.
- [22] D. Niizumi, D. Takeuchi, Y. Ohishi, N. Harada, and K. Kashino, “Masked Modeling Duo for speech: Specializing general-purpose audio representation to speech using denoising distillation,” in *Proc. Interspeech*, pp. 1294–1298, 2023.
- [23] W.-N. Hsu, B. Bolte, Y.-H. H. Tsai, K. Lakhotia, R. Salakhutdinov, and A. Mohamed, “HuBERT: Self-supervised speech representation learning by masked prediction of hidden units,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 29, pp. 3451–3460, 2021.
- [24] Y. Li, R. Yuan, G. Zhang, Y. Ma, X. Chen, H. Yin, C. Xiao, C. Lin, A. Ragni, E. Benetos, N. Gyenge, R. Dannenberg, R. Liu, W. Chen, G. Xia, Y. Shi, W. Huang, Z. Wang, Y. Guo, and J. Fu, “MERT: Acoustic music understanding model with large-scale self-supervised training,” in *Proc. International Conference on Learning Representations (ICLR)*, 2024.
- [25] A. Défossez, J. Copet, G. Synnaeve, and Y. Adi, “High fidelity neural audio compression,” *Transactions on Machine Learning Research*, 2023.
- [26] J. C. Brown, “Calculation of a constant Q spectral transform,” *The Journal of the Acoustical Society of America*, vol. 89, no. 1, pp. 425–434, 1991.
- [27] K. W. Cheuk, H. Anderson, K. Agres, and D. Herremans, “nnAudio: An on-the-fly GPU audio to spectrogram conversion toolbox using 1D convolutional neural networks,” *IEEE Access*, vol. 8, pp. 161981–162003, 2020.
- [28] Y. Li *et al.*, “MERT: official implementation (fairseq plug-in) and training scripts.” <https://github.com/yizhilll/MERT>. Accessed: October 2026.
- [29] m-a-p, “MERT-v0-public and MERT-v1-95m model cards.” <https://huggingface.co/m-a-p/MERT-v0-public>. Accessed: October 2026.
- [30] M. Won, Y.-N. Hung, and D. Le, “A foundation model for music informatics,” in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2024. arXiv:2311.03318.
- [31] C.-C. Chiu, J. Qin, Y. Zhang, J. Yu, and Y. Wu, “Self-supervised learning with random-projection

- quantizer for speech recognition,” in *Proc. International Conference on Machine Learning (ICML)*, *PMLR*, vol. 162, pp. 3915–3924, 2022.
- [32] H. Zhu, Y. Zhou, H. Chen, J. Yu, Z. Ma, R. Gu, Y. Luo, W. Tan, and X. Chen, “MuQ: Self-supervised music representation learning with Mel residual vector quantization.” arXiv preprint arXiv:2501.01108, 2025.
- [33] M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, and A. Joulin, “Emerging properties in self-supervised vision transformers,” in *Proc. IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 9630–9640, 2021.
- [34] D. Perera, V. Letzelter, T. Mariotte, A. Cortés, M. Chen, S. Essid, and G. Richard, “Annealed multiple choice learning: Overcoming limitations of winner-takes-all with annealing,” in *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 10541–10574, 2024.
- [35] S. Gururangan, A. Marasović, S. Swayamdipta, K. Lo, I. Beltagy, D. Downey, and N. A. Smith, “Don’t stop pretraining: Adapt language models to domains and tasks,” in *Proc. Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 8342–8360, 2020.
- [36] W.-N. Hsu, A. Sriram, A. Baevski, T. Likhomanenko, Q. Xu, V. Pratap, J. Kahn, A. Lee, R. Collobert, G. Synnaeve, and M. Auli, “Robust wav2vec 2.0: Analyzing domain shift in self-supervised pre-training,” in *Proc. Interspeech*, pp. 721–725, 2021.
- [37] S. Lee, M. Kang, J. Lee, S. J. Hwang, and K. Kawaguchi, “Self-distillation for further pre-training of transformers,” in *Proc. International Conference on Learning Representations (ICLR)*, 2023.
- [38] Y. Zhang, Y. Gao, W. Zhou, Z. Yuan, K. Imoto, and T. Kawahara, “SONAR: Self-distilled continual pre-training for domain adaptive audio representation.” arXiv preprint arXiv:2509.15703, 2025.
- [39] D. Niizumi, D. Takeuchi, M. Yasuda, B. T. Nguyen, Y. Ohishi, and N. Harada, “Towards pre-training an effective respiratory audio foundation model,” in *Proc. Interspeech*, 2025. arXiv:2505.15307.
- [40] Z. Li and D. Hoiem, “Learning without forgetting,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 12, pp. 2935–2947, 2018.
- [41] G. Hinton, O. Vinyals, and J. Dean, “Distilling the knowledge in a neural network.” arXiv preprint arXiv:1503.02531, 2015.
- [42] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell, “Over-

- coming catastrophic forgetting in neural networks,” *Proceedings of the National Academy of Sciences*, vol. 114, no. 13, pp. 3521–3526, 2017.
- [43] M. Defferrard, K. Benzi, P. Vandergheynst, and X. Bresson, “FMA: A dataset for music analysis,” in *Proc. International Society for Music Information Retrieval Conference (ISMIR)*, 2017.
- [44] NTT Communication Science Laboratories, “EVAR: Evaluation of audio representations.” <https://github.com/nttcs-lab/eval-audio-repr>. Accessed: October 2026.
- [45] R. Yuan, Y. Ma, Y. Li, G. Zhang, X. Chen, H. Yin, *et al.*, “MARBLE: Music audio representation benchmark for universal evaluation,” in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, vol. 36, 2023.
- [46] J. Turian, J. Shier, H. R. Khan, B. Raj, B. W. Schuller, *et al.*, “HEAR: Holistic evaluation of audio representations,” in *Proc. NeurIPS 2021 Competitions and Demonstrations Track, PMLR*, vol. 176, pp. 125–145, 2022.
- [47] B. L. Sturm, “The GTZAN dataset: Its contents, its faults, their effects on evaluation, and its future use.” arXiv preprint arXiv:1306.1461, 2013.
- [48] M. Defferrard *et al.*, “FMA: A dataset for music analysis, repository and download page.” <https://github.com/mdeff/fma>. Accessed: October 2026.
- [49] E. Fonseca, X. Favory, J. Pons, F. Font, and X. Serra, “FSD50K: An open dataset of human-labeled sound events,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 30, pp. 829–852, 2022.
- [50] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proc. International Conference on Learning Representations (ICLR)*, 2019.
- [51] M. Won, “MusicFM: code and pre-trained checkpoints.” <https://github.com/minzwon/musicfm>. Accessed: October 2026.
- [52] A. Queleennec, “MATPAC: Masked latent prediction and classification, code and checkpoints.” <https://github.com/aurianworld/matpac>. Accessed: October 2026.
- [53] C. Kereliuk, B. L. Sturm, and J. Larsen, “Deep learning and music adversaries,” *IEEE Transactions on Multimedia*, vol. 17, no. 11, pp. 2059–2071, 2015.
- [54] G. Tzanetakis and P. Cook, “Musical genre classification of audio signals,” *IEEE Transactions on Speech and Audio Processing*, vol. 10, no. 5, pp. 293–302, 2002.
- [55] J. Engel, C. Resnick, A. Roberts, S. Dieleman, M. Norouzi, D. Eck, and K. Simonyan, “Neural

- audio synthesis of musical notes with WaveNet autoencoders,” in *Proc. International Conference on Machine Learning (ICML)*, PMLR, vol. 70, pp. 1068–1077, 2017.
- [56] E. Law, K. West, M. Mandel, M. Bay, and J. S. Downie, “Evaluation of algorithms using games: The case of music tagging,” in *Proc. International Society for Music Information Retrieval Conference (ISMIR)*, pp. 387–392, 2009.
- [57] K. J. Piczak, “ESC: Dataset for environmental sound classification,” in *Proc. ACM International Conference on Multimedia*, pp. 1015–1018, 2015.
- [58] J. Salamon, C. Jacoby, and J. P. Bello, “A dataset and taxonomy for urban sound research,” in *Proc. ACM International Conference on Multimedia*, pp. 1041–1044, 2014.
- [59] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1993.

Chapter A First Appendix if Required

If you want to add appendices then make sure you provide a proper title. Add different appendices by using the chapter command as shown for this one. They will automatically be added to the table of contents.

A.1 References

See how .bib file is prepared. The bibliography would be automatically added by Latex. So here is how you cite papers (you'll need to view the .bib file to understand). [3] is the correct way to make one citation and [2, 4] to make multiple citations. You can see various reference styles for manuals [4], conferences [5], websites [6] and journal articles [1]. Also, note that all citations appear in the order in which they appear within text. They are not sorted according to author names.

A.2 Equations

Make sure you use the math environment for writing equations. Again, make sure you do not hard code equation numbers. Use label and ref commands of Latex to number them and reference them. Here is an example of (A.1)

$$E = mc^2 \tag{A.1}$$

Note, how the equation is referenced by its equation number within brackets.

A.3 Figures

To insert a figure, you can use the following code as template. Here the figure is referenced using ref command of Latex. So Figure A.1 is the FAST logo. DO NOT HARD CODE FIGURE NUMBERS. Let Latex manage them for you.



Figure A.1: Fast Logo is the Caption.

A.4 Tables

Note how tables can be added in Latex. Also, make sure you use label and ref commands to define and use table numbers. You can use Table A.1 as a template to make tables, add captions to them and reference them. DO NOT HARD CODE TABLE NUMBERS. Let Latex manage their numbering for you.

If you are having problems in managing Latex tables, you may use Lyx <https://www.lyx.org/>.

Table A.1: Give a Caption to the Table.

No.	Column 1	Column 2
1.	Some data	More data
2.	Some values	More values

A.5 Pseudo Code

When you need to explain an existing algorithm or an algorithm you devised, then you should use pseudo code notation. Given below, algorithm 3, is an example. Note how the numbering is managed by the Latex itself.

Data: this text

Result: how to write algorithm with L^AT_EX2_ε

initialization;

while *not at end of this document* **do**

 read current;

if *understand* **then**

 go to next section;

 current section becomes this one;

else

 go back to the beginning of current section;

end

end

Algorithm 3: How to write algorithms

A.6 Code of Programming Languages

If you need to add C++, Java or any other programming language's code, then you can use "listings" package. Following is an example. Normally, the code is only added in the appendices to avoid clutter

in the document.

```
#include <stdio.h>
#define N 10
/* Block
   * comment */

int main()
{
    int i;

    // Line comment.
    puts("Hello world!");

    for (i = 0; i < N; i++)
    {
        puts("LaTeX is also great for programmers!");
    }

    return 0;
}
```

A.7 Recommendations for L^AT_EX

Here we are providing you with only two Latex source files. main.tex and fypbib.bib file. You can also start a project at overleaf.com and upload the zip file of this template there and share the Latex source between your group and supervisor. This way all of you can work on the Latex files together online.